Vietnam General Confederation of Labor

Ton Duc Thang University

Faculty of Information Technology



Dinh Quoc Cuong – 523H0008

Vo Huynh Minh Duc – 523H0014

Building a Video Sharing Platform with an Automated HLS Transcoding Pipeline on a Serverless Container and Event-Driven Architecture

INFORMATION TECHNOLOGY PROJECT

DEVOPS, CONTAINERIZATION & CLOUD SYSTEMS

Supervisor

Msc. Mai Van Manh

Ho Chi Minh City, 2026

Acknowledgment

First and foremost, we would like to express our sincere gratitude to our supervisor, **MSc. Mai Van Manh**, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this project. His expertise, patience, and dedication have played a crucial role in shaping both the technical direction and the academic rigour of this work.

We are also deeply thankful to the lecturers and staff of the Faculty of Information Technology at Ton Duc Thang University, whose support and resources provided a strong foundation for our work. Their commitment to creating an inspiring and intellectually stimulating environment has greatly contributed to our growth.

Our gratitude further extends to our classmates and friends who offered helpful discussions, motivation, and meaningful insights during the development and evaluation of this project. Their companionship made the challenging moments far more manageable.

Finally, we are profoundly grateful to our families for their unconditional support, patience, and encouragement. Their belief in us has always been our greatest source of strength and determination.

To all of you, we extend our heartfelt thanks.

Certification

This thesis was carried out at **Ton Duc Thang University**.

Advisor: MSc. Mai Van Manh

.....
(Title, Full Name and Signature)

This thesis was defended at the Undergraduate Thesis Examination Committee held at Ton Duc Thang University on . . . . . / . . . . . / . . . . . .

Confirmation of the Chairman of the Undergraduate Thesis Examination Committee and the Dean of the Faculty after receiving the revised thesis (if any).

CHAIRMAN **DEAN OF FACULTY**

. . . . . .

Declaration

This thesis was completed at **Ton Duc Thang University**. We hereby declare that this is our own research work, conducted under the academic supervision of **MSc. Mai Van Manh**. All analyses, discussions, and results presented in this thesis are truthful, original, and have not been published in any form prior to this submission.

All data used for analysis, evaluation, and comparison were collected directly by the author(s) from various reliable sources, which are fully cited in the reference section of this thesis. In addition, certain evaluations, comments, and figures quoted from other authors, organizations, or research works are clearly referenced and properly acknowledged.

If any form of academic dishonesty, plagiarism, or copyright infringement is discovered, we accept full responsibility for the content of this thesis. Ton Duc Thang University shall bear no liability for any copyright or authorship violations committed by the author(s) during the execution of this research.

Ho Chi Minh City, day month year

Author(s)

(Signature and full name)

Abstract

Modern video sharing platforms must solve a common core problem: when a user uploads a source file in an arbitrary format, codec, or resolution, the system has to transcode it automatically into HTTP Live Streaming (HLS) so that browsers can play it with adaptive quality. Provisioning a dedicated server cluster or a Kubernetes system that runs continuously purely to wait for transcoding work is wasteful, because upload traffic is inherently bursty and the infrastructure sits idle most of the time. Conversely, when many users upload simultaneously, the system must scale out to process hundreds of videos in parallel.

This project presents the design, implementation, and evaluation of a complete video sharing platform whose transcoding subsystem is built on a Serverless Container and Event-Driven architecture. Uploading a file to Amazon S3 emits an event that is buffered in Amazon SQS and dispatched by an AWS Lambda job submitter to AWS Batch running on AWS Fargate. A container packaging FFmpeg is created only when there is work to do, produces three renditions at 360p, 720p, and 1080p together with a master playlist and a thumbnail, uploads the result to a processed bucket, updates the metadata in MongoDB Atlas, and then terminates. Content is delivered through Amazon CloudFront with Origin Access Control, and every stream is gated at the edge by CloudFront Signed Cookies, so that a viewer without a valid signature is rejected even when they know the exact manifest URL; a video's visibility setting governs who the backend will issue those cookies to, not whether the gate applies.

The entire AWS infrastructure is described as code using eleven independent Terraform modules and can be provisioned with a single command. Seven GitHub Actions workflows automate linting, unit testing, static security analysis, container image scanning, infrastructure validation, and deployment, with a two-tier quality gate that blocks a pull request whenever a fixable critical vulnerability is detected.

The system is evaluated along four dimensions: quality of experience, measured both as Time-to-First-Frame and, by driving the deployed player in a headless browser under emulated network conditions, as rebuffering ratio, bitrate switching and the bitrate a

viewer actually receives; transcoding pipeline latency across 100 MB, 500 MB, and 1 GB inputs, scalability under a concurrent upload stress test, and operating cost compared against a traditional always-on Amazon EC2 deployment. The cost analysis shows that, for the architecture-dependent portion of the bill, the serverless approach removes almost all idle resource waste, although this advantage narrows as sustained utilisation increases.

List of Abbreviations

ABR	Adaptive Bitrate Streaming
ACM	AWS Certificate Manager
API	Application Programming Interface
ARN	Amazon Resource Name
AWS	Amazon Web Services
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DLQ	Dead Letter Queue
DNS	Domain Name System
DRM	Digital Rights Management
EBS	Amazon Elastic Block Store
ECR	Amazon Elastic Container Registry
ECS	Amazon Elastic Container Service
ERD	Entity Relationship Diagram
FinOps	Cloud Financial Operations
FR	Functional Requirement
GPU	Graphics Processing Unit
HEVC	High Efficiency Video Coding
HLS	HTTP Live Streaming
IaC	Infrastructure as Code
IAM	AWS Identity and Access Management
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
NFR	Non-Functional Requirement
OAC	Origin Access Control

QoE	Quality of Experience
SAST	Static Application Security Testing
SCA	Software Composition Analysis
SPA	Single Page Application
SQS	Amazon Simple Queue Service
S3	Amazon Simple Storage Service
TLS	Transport Layer Security
TTFF	Time-to-First-Frame
TTL	Time To Live
UC	Use Case
vCPU	Virtual Central Processing Unit
VMAF	Video Multi-Method Assessment Fusion
VOD	Video on Demand
VPC	Amazon Virtual Private Cloud

Contents

Acknowledgment **i**

Certification **ii**

Declaration **iii**

Abstract **iv**

List of Abbreviations **vi**

1 Introduction **1**

1.1 Background and Rationale 1

1.2 Project Objectives 2

1.3 Scope and Deliverables 2

1.4 Research Methodology 3

1.5 Practical Significance 3

1.6 Main Contributions 4

1.7 Structure of the Report 4

2 Literature Review and Technologies **6**

2.1 Related Platforms and Comparison 6

2.2 Core Models and Principles 8

2.2.1 Serverless Containers 8

2.2.2 Event-Driven Architecture 10

2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder 10

2.3 Technology Stack Justification 12

3 Requirements Analysis **13**

3.1 Stakeholders and User Requirements 13

3.2 Functional Requirements 13

3.3 Non-Functional Requirements 15
3.4 Use Case Model 15

4 System Design **18**
4.1 High-Level Architecture 18
4.2 Transcoding Pipeline Design 18
4.3 Database Design 22
4.4 Content Protection Design 24
4.5 Interface Design 25

5 Development and Testing **28**
5.1 Backend Implementation 28
5.2 Transcoder Implementation 29
5.3 Infrastructure as Code 30
5.4 CI/CD Pipeline and DevSecOps 31
5.5 Challenges During Development 33
5.5.1 Provisioning Blocked by Account Policy 33
5.5.2 A Signed Cookie Policy That Matched Nothing 34
5.5.3 Two Consequences of Gating Every Object 34
5.5.4 Playback Across the Cookie Expiry Boundary 35
5.5.5 An Empty Secret That Stopped the Pipeline 35
5.5.6 Retry and Alerting on the Deployed Path 35
5.5.7 Monitoring the Service Rather Than the Server 36
5.5.8 Infrastructure Drift in the Instance Definition 36
5.6 Testing 37

6 Deployment and Evaluation **40**
6.1 The Deployed Environment 40
6.2 User Interface Showcase 41
6.3 Quality of Experience Evaluation 44
6.3.1 Measuring the Deployed CDN, and a Configuration That Inverted It 44
6.3.2 Delivery Latency by Client Location 46
6.3.3 Playback Under Constrained Bandwidth, and a Manifest That Understated Itself 47
6.3.4 Whether Content Complexity Governs the Result 51
6.4 Transcoding Pipeline Performance 54
6.5 Chunked Transcoding of Long Videos 56

6.6 Load Testing 61
6.6.1 Cross-Validation with an Independent Tool 62
6.6.2 Validating Real Transcoding Throughput 63
6.6.3 Drain Rate Under Real Transcoding Load 64
6.7 Cost Analysis 65

7 Conclusion and Future Work **67**
7.1 Summary of Achievements 67
7.2 Strengths of the System 68
7.3 Limitations 69
7.4 Future Work 71

A Supplementary Material **75**
A.1 Repository Structure 75
A.2 FFmpeg Transcoding Command 75
A.3 Signed Cookie Custom Policy 77
A.4 Reproducing the Measurements 78
A.5 Deploying the Infrastructure 79

List of Figures

2.1 *Compute models for a bursty transcoding workload: an always-on cluster compared with on-demand serverless containers* 9
2.2 *HLS artefacts produced from one source file and the order in which a player retrieves them* 11

3.1 *Use case diagram of the video sharing platform* 16

4.1 *Overall system architecture, showing the deployment boundaries and the numbered data flow from an anonymous visit through upload, transcoding and signed delivery* 19
4.2 *Sequence diagram of the event-driven transcoding pipeline* 20
4.3 *Entity relationship diagram of the MongoDB data model* 23
4.4 *User flow through the application, from an anonymous visit to a published video* 26

5.1 *The seven GitHub Actions workflows, the file paths that trigger each of them, and the two-tier DevSecOps quality gate* 32

6.1 *Catalogue page, the entry point for a visitor* 41
6.2 *Watch page with custom player, rendition selector and related videos sidebar* 42
6.3 *Upload page, showing direct-to-storage transfer with live telemetry* . . . 43
6.4 *Channel management view, including videos still being processed* . . . 43

List of Tables

2.1 Comparison of transcoding approaches across platforms 7

3.1 Functional requirements of the system 14
3.2 Non-functional requirements of the system 15
3.3 Use case specification: upload a video 17

5.1 Terraform modules and their responsibilities 30
5.2 Automated test coverage by area 38

6.1 Time-to-First-Frame measurement results 44
6.2 Time-to-First-Frame through CloudFront after the price class correction 45
6.3 Time-to-First-Frame by client location, before and after `PriceClass_All` 46
6.4 Playback quality by emulated network profile 48
6.5 Declared against actual bitrate for each rendition 49
6.6 Playback quality before and after the manifest correction 50
6.7 Playback quality across three sources of differing visual complexity . . 52
6.8 Unthrottled playback at device pixel ratio one against two 53
6.9 Transcoding pipeline latency by input size 54
6.10 Single-container path against chunked path for videos of five to fifteen minutes 57
6.11 A 25-minute video on the chunked path, by chunk configuration and capacity 58
6.12 A four-hour video at two capacities 59
6.13 Four 25-minute videos submitted together, by capacity 59
6.14 Simultaneous submissions at 16 vCPU 60
6.15 Concurrent upload stress test results (API layer) 62
6.16 Cross-validation of the load test across two independent tools 62
6.17 Backlog drain under synthetic and real payloads 65
6.18 Monthly operating cost comparison (ap-southeast-1) 66

A.1 Provenance of each measurement table 78

Chapter 1

Introduction

1.1 Background and Rationale

Video has become the dominant form of content on the Internet, and platforms such as YouTube, Vimeo, and TikTok have made uploading and sharing video an everyday activity. Behind that apparent simplicity lies a demanding engineering problem. When a user uploads a source file, that file may use any container format, any codec, and any resolution, yet the platform must deliver a stream that plays smoothly on every device and adapts to whatever bandwidth the viewer happens to have at that moment.

The industry solution to this problem is HTTP Live Streaming (HLS), a protocol that splits a video into short segments, typically between two and ten seconds long, and produces several quality renditions of the same content. A playlist file written in the `.m3u8` format enumerates the available renditions, and the player selects among them in real time according to measured network throughput. This mechanism, known as Adaptive Bitrate Streaming (ABR), is what allows playback to degrade gracefully rather than stall when a connection weakens.

Producing those renditions is computationally expensive, and this is where the architectural difficulty arises. Transcoding is a bursty workload: it consumes a large amount of CPU while a video is being processed and nothing at all when no one is uploading. A platform that keeps a dedicated server cluster or a Kubernetes system running continuously simply to wait for work therefore pays for a substantial amount of idle capacity. The alternative failure mode is equally undesirable: a system provisioned only for the average case cannot absorb a sudden burst when many users upload at the same time, and uploads either queue for a long time or are dropped altogether.

This tension between idle cost and burst capacity motivates the present project. Rather than maintaining permanently allocated compute, the system proposed here starts

a container only when there is a video to process and destroys it immediately afterwards, so that cost accrues only for the seconds during which transcoding actually runs.

1.2 Project Objectives

The project aims to build a complete video sharing platform for end users, together with the automated transcoding subsystem that supports it. Specifically, the objectives are:

1. To build a web platform that allows users to register an account, upload videos, watch them at multiple quality levels, and share them through public, unlisted or private links.

2. To automate the entire processing pipeline, from the moment a source file lands in Amazon S3 through transcoding, storage, and delivery via a content delivery network, until the video becomes ready for playback in a browser.

3. To perform transcoding with FFmpeg inside a Docker container running on AWS Batch and AWS Fargate, such that containers are created on demand and terminate automatically, thereby eliminating the cost of idle resources.

4. To automate software deployment and infrastructure management through a CI/CD pipeline and Infrastructure as Code.

5. To evaluate the resulting system empirically in terms of quality of experience, transcoding performance, scalability under load, and operating cost.

1.3 Scope and Deliverables

The scope of this project covers the full vertical slice of a video sharing service: the browser-based user interface, the backend API, the transcoding engine, the cloud infrastructure that hosts them, and the automation that deploys them.

Several areas are deliberately placed outside the scope. The project does not implement digital rights management systems such as Widevine or FairPlay, which would be required to prevent an authorised viewer from redistributing content they can legitimately access. It does not implement live streaming, since the architecture is designed around video on demand. It also does not attempt per-title bitrate ladder optimisation, using instead a fixed ladder of three renditions; the rationale and the trade-off involved are discussed in Chapter 7.

1.4 Research Methodology

The work combines a review of existing practice with an iterative build-and-measure cycle.

The theoretical component established how the problem is solved at industrial scale before any code was written. Published engineering accounts from Netflix describe how a large platform decomposes video processing into microservices and how a bitrate ladder is optimised per title, and the AWS Video on Demand reference architecture describes the managed-service equivalent. The protocol itself was taken from its specification, RFC 8216, and the encoding parameters from the FFmpeg HLS muxer documentation, rather than from secondary sources. Peer-reviewed studies of serverless video processing supplied both the motivation for the architecture and the measurement vocabulary used later in the evaluation, and the multi-threaded H.264 literature supplied the expected bound against which the vCPU scaling results of Chapter 6 are interpreted.

The practical component followed an incremental development model. Each subsystem, comprising the backend API, the transcoding container, the frontend, and the cloud infrastructure, was built and integrated in turn, with the whole environment described as Terraform code so that every change to it is reviewable and reproducible rather than applied by hand. Correctness was checked by an automated test suite executed on every push through a continuous integration pipeline, which also runs dependency, container and infrastructure security scans.

Evaluation was empirical rather than analytical. Every performance figure reported in Chapter 6 was produced by a measurement script committed alongside the source code, executed against the deployed system rather than a local approximation of it, with the raw output stored as JSON under `docs/results/` so that any table in this report can be traced back to the run that produced it; Table A.1 records that mapping. Where a result could plausibly be an artefact of the measurement tool, it was reproduced with a second, independently implemented tool before being reported.

1.5 Practical Significance

The immediate beneficiaries of this design are small teams and individual creators who need a video platform but cannot justify a permanently running transcoding cluster. The cost comparison in Section 6.7 estimates that processing three hundred videos a month costs roughly $1.51 on the architecture built here against $133.35 on an always-on instance sized for the same work, because compute exists only while a job runs. That

difference is what makes video hosting feasible for a service whose upload volume is intermittent, which describes most platforms before they become large.

The design also addresses a requirement that is easy to state and easy to implement incorrectly. Marking a video private is meaningless if the underlying storage remains publicly addressable, and Chapter 4 shows that restricting the API alone does not achieve it. The two-layer scheme used here, keeping the bucket private and enforcing authorisation at the content delivery network, is directly reusable by any application that serves user-owned media from object storage.

Beyond the product itself, the project serves as a worked reference for an event-driven serverless pipeline on AWS, covering the parts that are usually omitted from tutorials: how duplicate message delivery is made harmless, how the infrastructure is expressed as reviewable code across two environments, how security scanning is placed in the path of a merge rather than beside it, and how the resulting system is measured once it is running. Each of these is documented here in enough detail to be repeated.

1.6 Main Contributions

The principal contribution of this work is an end-to-end demonstration that a serverless container architecture can serve as the transcoding backbone of a video platform, together with an empirical assessment of what that choice costs and what it saves.

More concretely, the work contributes an event-driven pipeline in which an upload event propagates through Amazon S3, Amazon SQS, and AWS Lambda to AWS Batch without any polling service in between; a containerised FFmpeg worker whose conditional database writes make a duplicate delivery harmless, so that exactly-once *effects* are obtained without relying on exactly-once delivery; a two-layer content protection scheme combining Origin Access Control with CloudFront Signed Cookies so that private videos remain inaccessible even to a viewer holding the exact manifest URL; a complete Infrastructure as Code description of the AWS environment in eleven Terraform modules; and a DevSecOps pipeline whose security gate genuinely blocks merges rather than merely reporting findings.

1.7 Structure of the Report

The remainder of this report is organised as follows. Chapter 2 reviews how comparable platforms solve the transcoding problem and justifies the technology choices made here. Chapter 3 analyses the functional and non-functional requirements of the system and

presents the use case model. Chapter 4 describes the architecture, the data model, the content protection scheme, and the interface design. Chapter 5 details the implementation of each subsystem, the infrastructure automation and continuous integration, the difficulties encountered during development and how they were resolved, and the automated test suite. Chapter 6 presents the deployed system and its user interface, then reports the experimental results for delivery latency, transcoding performance, behaviour under load, and operating cost. Chapter 7 summarises what was achieved, states the strengths and limitations of the delivered system, and outlines directions for future work.

Chapter 2

Literature Review and Technologies

2.1 Related Platforms and Comparison

Understanding how established platforms solve the transcoding problem provides the necessary context for the design decisions taken in this project.

Netflix offers the most thoroughly documented case. Its original video processing system, internally known as Reloaded, was a monolithic pipeline that proved difficult to evolve as requirements changed. Netflix subsequently rebuilt it as a set of independent microservices running on a platform called Cosmos, decomposing the workload into a Video Inspection Service, a Video Encoding Service, and a Video Quality Service [1]. Cosmos itself is layered: an API tier receives requests, a workflow tier orchestrates the processing steps, and a serverless compute tier performs the actual media work, with the three tiers communicating asynchronously through a priority-based messaging system [2]. Two aspects of this design are directly relevant here. First, Netflix also relies on asynchronous messaging rather than synchronous calls to decouple job submission from job execution, which is precisely the role played by Amazon SQS in this project. Second, Netflix moved from linear encoding to distributed chunk-based encoding, allowing hundreds of machines to encode segments of the same title in parallel and reducing processing latency from days to hours. This project applies the same principle at a far smaller scale: for videos of five minutes or longer, a planner job divides the source into chunks that an array of AWS Batch jobs encodes concurrently and a finalizer job merges, with the degree of parallelism bounded by the Fargate vCPU quota rather than by a fleet of dedicated machines (Section 6.5).

Amazon Web Services publishes a reference architecture for video on demand that provides a useful point of comparison at the level of managed services [3]. That architecture routes an upload from Amazon S3 through an AWS Lambda job submitter into AWS Elemental MediaConvert, stores the output in a second S3 bucket, and delivers it through Amazon CloudFront, with AWS Step Functions orchestrating the workflow. The architecture presented in this report follows the same overall shape but substitutes a self-packaged FFmpeg container on AWS Batch for MediaConvert. This substitution is the central engineering trade-off of the project: MediaConvert is a fully managed service billed per minute of output and requires almost no operational effort, whereas a self-managed container is considerably cheaper per unit of work but transfers the responsibility for image maintenance, dependency updates, and failure handling onto the development team. AWS documentation also confirms that Origin Access Control is the currently recommended pattern for securing an S3 origin, superseding the legacy Origin Access Identity mechanism, which validates the approach adopted in Section 4.4.

Table 2.1 summarises the comparison across the dimensions most relevant to this work.

Table 2.1: Comparison of transcoding approaches across platforms

ASPECT	**NETFLIX (COSMOS)**	**AWS VOD REFERENCE**	**THIS PROJECT**
Compute model	Dedicated microservice platform with serverless compute tier	Managed service (MediaConvert)	Serverless container on AWS Batch / Fargate
Parallelism	Chunk-based, distributed across many nodes	Managed internally by the service	Chunk-based on AWS Batch array jobs from five minutes of video, one container below that
Bitrate ladder	Per-shot optimisation driven by VMAF	Configurable job template	Fixed ladder: 360p, 720p, 1080p
Queueing	Priority-based messaging system	AWS Step Functions	Amazon SQS with dead letter queue
Content protection	DRM	Signed URLs or signed cookies	OAC plus CloudFront Signed Cookies
Operational effort	Very high	Very low	Moderate

2.2 Core Models and Principles

2.2.1 Serverless Containers

The term serverless container describes a model in which a container image is executed without the developer provisioning or managing the underlying virtual machines, and in which billing is based on execution time rather than allocated capacity. AWS Fargate is the implementation used in this project.

Figure 2.1 contrasts this model with the conventional alternative for a transcoding workload of the size assumed in Section 6.7. In the always-on arrangement on one side of the figure, a cluster is sized for the busiest moment the service expects and then runs continuously, so it is billed for all 730 hours in a month while performing roughly 3.75 hours of actual encoding, a utilisation of about half a percent. Its capacity is also a hard ceiling: an upload burst larger than the cluster was provisioned for must either wait or be rejected, and the only remedy is to provision for a still larger peak and waste correspondingly more. In the serverless arrangement on the other side, no compute exists between uploads. An upload event enters a queue, a container is started to serve it, and that container terminates when the job finishes, so charges accrue per vCPU-second of real work. The burst is absorbed by the queue rather than by spare capacity, and the degree of parallelism is a configured number rather than a fixed fleet size. The two figures shown, $124.10 against $0.05 of monthly compute, are the same workload priced under the two models.

The choice of Fargate over a function-as-a-service platform such as AWS Lambda deserves justification, because Lambda is the more common answer to serverless workloads. Video transcoding is unsuitable for Lambda in this context for two reasons. The first is duration: transcoding a large file routinely exceeds the maximum execution time of a Lambda invocation. The second concerns resource configuration. A measurement study of serverless video processing found that selecting a cost-efficient memory configuration for transcoding is far from trivial, because the optimal setting depends not only on the type of task but even on the content of the video itself, and the cost difference between the least and most generous memory configuration reached 498% for sequential workloads [4]. Fargate avoids this difficulty by allowing vCPU and memory to be specified explicitly and independently.

A known weakness of the container model is cold start latency. A systematic review of the subject reports that more than half of function invocations spend at least 80% of their startup time pulling the container image [5]. This finding directly motivates the use of a multi-stage Docker build in this project, described in Section 5.2, whose purpose is

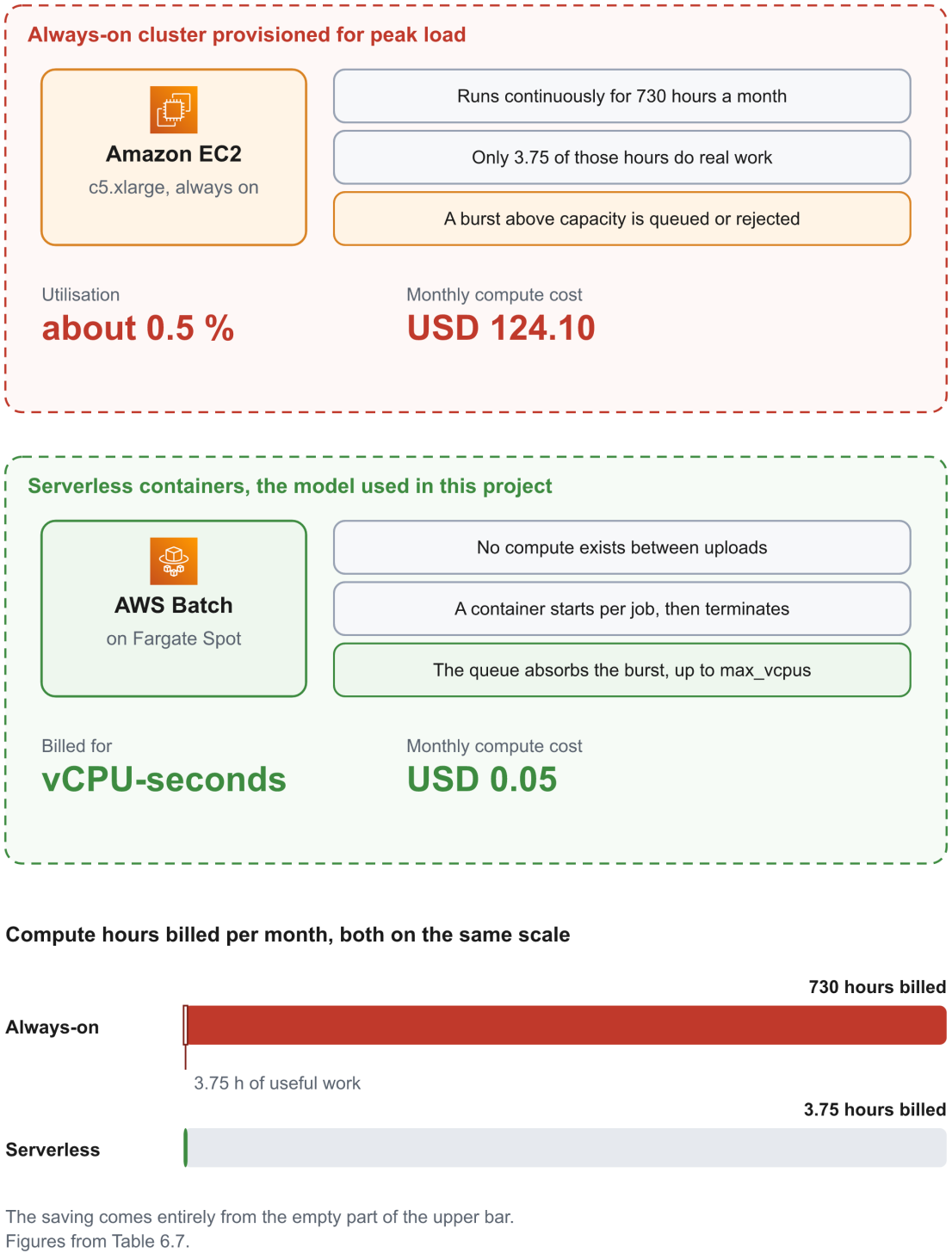


Figure 2.1: *Compute models for a bursty transcoding workload: an always-on cluster compared with on-demand serverless containers*

to minimise the size of the image that must be transferred before work can begin.

2.2.2 Event-Driven Architecture

An event-driven architecture propagates work through the emission and consumption of events rather than through direct synchronous invocation. In this project, the completion of an upload to Amazon S3 emits an event notification that is placed on an Amazon SQS queue, from which an AWS Lambda function submits a job to AWS Batch.

The queue is not merely a routing mechanism; it is the component that makes burst absorption possible. If a hundred users upload simultaneously, the queue retains all hundred messages while AWS Batch works through them at whatever rate the compute environment allows. Without this buffer, the system would have to either reject work or provision for peak load permanently. The queue is paired with a dead letter queue so that messages which repeatedly fail processing are isolated for inspection rather than being retried indefinitely.

2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder

HLS delivers a video as a master playlist referencing several rendition playlists, each of which lists the media segments for one quality level. The player measures throughput and switches between renditions at segment boundaries.

Figure 2.2 shows the artefacts this produces and the order in which a player retrieves them. A single source file of arbitrary container, codec and resolution is encoded once per rung of the ladder, and each rung yields both a rendition playlist and the sequence of transport stream segments it indexes. The master playlist sits above them and carries, for each rung, the bandwidth the client should expect to need in order to play it. A player therefore begins by fetching only the master playlist, which is a few hundred bytes, chooses a rung from the declared bandwidths and its own throughput estimate, and only then requests that rung's playlist and its segments. Because every rung is segmented on the same boundaries, the player may change its choice at any segment boundary without interrupting playback, which is the mechanism that lets quality fall rather than the video stall when a connection weakens. The consequence for the transcoding subsystem is that the cost of preparing a video scales with the number of rungs, since each rung is encoded separately even though the source itself is decoded only once. The implementation exploits exactly that distinction, as Section 5.2 describes.

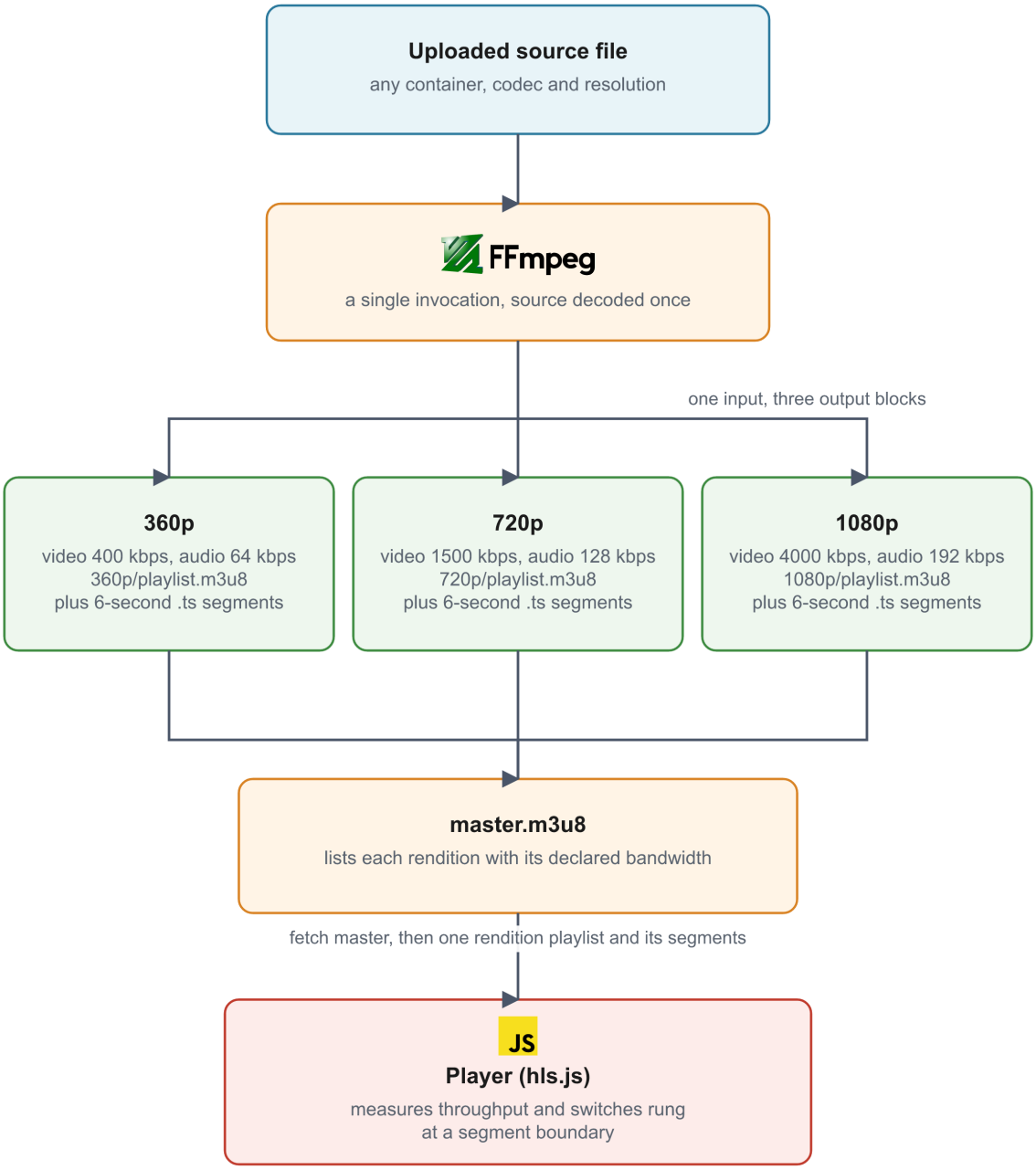


Figure 2.2: *HLS artefacts produced from one source file and the order in which a player retrieves them*

The set of resolution and bitrate pairs offered is called the bitrate ladder, and its construction is an active research area. Netflix demonstrated that the optimal ladder is not fixed but content-dependent, and that the ideal operating points lie on the convex hull of the quality-versus-bitrate curve [6]. By encoding each shot separately under the control of the VMAF perceptual quality metric, Netflix reported bitrate reductions of between 28% and 38% at equivalent visual quality [7]. The present project uses a fixed

three-rung ladder, which is simpler to implement and reason about; the implications of this simplification are revisited in Section 7.4.

2.3 Technology Stack Justification

The frontend is implemented as a React single page application built with Vite, using HLS.js for playback. HLS.js is necessary because most desktop browsers do not support HLS natively; it parses the manifest in JavaScript and feeds segments to the browser through the Media Source Extensions API. Safari, which does support HLS natively, is handled through a fallback path.

The backend is built on Node.js with Express. The decisive factor is that the backend's workload is almost entirely input/output bound rather than CPU bound: it issues pre-signed URLs, queries MongoDB, and returns JSON, while the heavy computation is deliberately delegated to the transcoder containers. Node.js's event loop is well suited to this profile.

MongoDB Atlas provides the persistence layer. Video metadata is naturally document-shaped and its schema evolves as features are added, which suits a document store better than a rigid relational schema. Using the managed Atlas service also removes the need to operate a database cluster inside the project's own infrastructure.

Terraform was selected for Infrastructure as Code over alternatives such as AWS CloudFormation because its module system allows the infrastructure to be decomposed into reusable units with explicit inputs and outputs, and because its plan step makes the consequences of a change visible before it is applied. GitHub Actions was selected for CI/CD because the source code is already hosted on GitHub, which removes the need for a separate automation server.

Chapter 3

Requirements Analysis

3.1 Stakeholders and User Requirements

Three groups interact with the system, and their differing needs shape the requirements.

The *anonymous visitor* arrives without an account. This group needs to browse public videos, search and filter by category, and watch content immediately, because requiring registration before any content can be seen would deter adoption. The *registered user* additionally needs to upload videos, control whether each video is public, unlisted or private, manage the videos on their own channel, and interact through likes and comments. The *system administrator*, in the context of this project the development team itself, needs the platform to be observable and deployable without manual intervention.

3.2 Functional Requirements

Table 3.1 enumerates the functional requirements derived from the stakeholder analysis. Each requirement is given an identifier so that it can be traced through the design and implementation chapters.

Table 3.1: Functional requirements of the system

ID	REQUIREMENT	DESCRIPTION
FR-01	Registration and login	Users register with a username, email address, and password. Authentication uses JSON Web Tokens carried in the Authorization header.
FR-02	Password recovery	Users request a reset link by email; the token is single-use and expires after fifteen minutes.
FR-03	Direct upload to S3	The client requests a pre-signed URL from the backend and transfers the file directly to Amazon S3 without routing the payload through the application server.
FR-04	Automatic transcoding	Once the upload completes, the system transcodes the source into HLS renditions without any manual trigger.
FR-05	Adaptive playback	The player switches automatically among 360p, 720p, and 1080p according to available bandwidth.
FR-06	Visibility control	Each video is public, unlisted, or private. A public video is listed in the catalogue; an unlisted one is reachable only by its direct link and appears in no listing; a private one is visible to its owner alone. The owner can change the setting after upload.
FR-07	Channel management	Users view their own uploads, manage visibility, update video details, and delete a video together with all of its stored artefacts.
FR-08	Search and filtering	Users search by title, description, or tags, and filter by category.
FR-09	Engagement	Users express likes and dislikes and post comments; a comment may be removed by its author or by the video owner.
FR-10	View counting	The system records a view only after playback has genuinely started, and does not count repeated views from the same viewer within a short window.
FR-11	Related videos	The watch page suggests further videos drawn from the same category and the same uploader. A suggestion is subject to the same visibility rule as the video itself, so the list never reveals one the viewer could not open directly.

Requirement FR-03 deserves particular comment because it shapes the architecture more than any other functional requirement. Routing a multi-gigabyte upload through the backend would consume server memory and bandwidth proportional to the number of concurrent uploads and would make the API server the bottleneck of the entire platform. Issuing a time-limited pre-signed URL and letting the browser transfer the file directly to Amazon S3 removes that bottleneck entirely: the backend handles only a small JSON request regardless of file size.

3.3 Non-Functional Requirements

The non-functional requirements are stated in measurable terms wherever possible, so that Chapter 6 can report whether they were met.

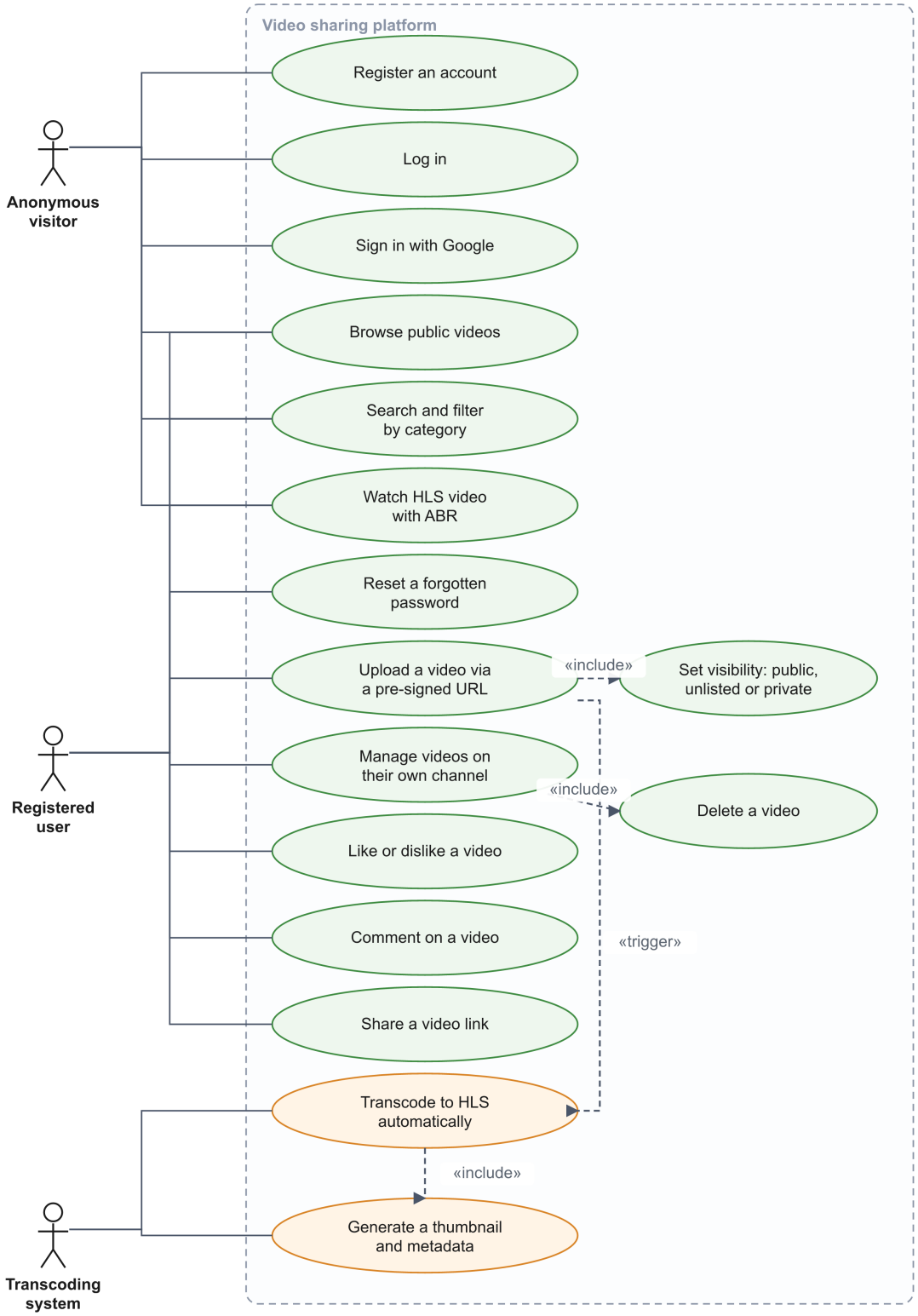
Table 3.2: Non-functional requirements of the system

ID	ATTRIBUTE	REQUIREMENT
NFR-01	Latency	Time-to-First-Frame should remain low enough that playback feels immediate; the measurement methodology is defined in Section 6.3.
NFR-02	Scalability	The pipeline must absorb a burst of concurrent uploads without losing work, processing them as capacity permits rather than rejecting them.
NFR-03	Reliability	A failed transcoding job must not silently disappear: it must be retried where retrying could succeed, and must raise an alert carrying the reason when it cannot. The retry and alerting mechanisms that satisfy this requirement on the deployed path are described in Section 5.5.
NFR-04	Cost efficiency	Compute cost must accrue only during actual transcoding, with no charge for idle capacity.
NFR-05	Security of content	Private videos must be inaccessible to unauthorised viewers even when the exact manifest URL is known.
NFR-06	Security of the API	Authentication endpoints must resist brute-force attempts, and the origin of cross-origin requests must be restricted to an explicit allow-list.
NFR-07	Reproducibility	The entire infrastructure must be reproducible from source control through a single command.
NFR-08	Maintainability	Changes must pass automated tests and security scanning before reaching the main branch.

Requirement NFR-05 is worth distinguishing from ordinary access control. Restricting an API response is straightforward, but a video is not delivered by the API: it is delivered by a content delivery network as hundreds of individual segment files. An access control scheme that protects only the API therefore leaves the actual content unprotected, a gap that Section 4.4 addresses explicitly.

3.4 Use Case Model

Figure 3.1 presents the use case diagram of the system. Three actors appear: the anonymous visitor, the authenticated user, and the transcoding system itself, which is modelled as an actor because it initiates processing autonomously in response to events rather than in response to a direct user action.



Association lines carry no arrowhead, as UML specifies. Only «include» and «trigger» relationships are directed. The transcoding use cases attach to the system actor because events start them, not a human action.

Figure 3.1: *Use case diagram of the video sharing platform*

The diagram makes visible an important property of the design: the transcoding use cases are not invoked by the user at all. The user's upload action triggers them indirectly through the event chain, which is why they are attached to the system actor rather than to the human actors. This separation is what allows the browser to return control to the user immediately after the upload completes, without waiting for processing to finish.

Table 3.3 specifies the most architecturally significant use case in detail.

Table 3.3: Use case specification: upload a video

FIELD	**CONTENT**
Identifier	UC-07
Actor	Authenticated user
Precondition	The user holds a valid JSON Web Token.
Trigger	The user selects a video file and submits the upload form.
Main flow	1. The client validates the file type and size locally. 2. The client calls the initiate-upload endpoint with the file metadata. 3. The server validates the metadata, creates a database record with status `UPLOADING`, and returns a pre-signed URL valid for fifteen minutes. 4. The browser transfers the file directly to the raw S3 bucket. 5. The client confirms completion and the record moves to `PROCESSING`. 6. The transcoding pipeline runs asynchronously and eventually sets the record to `READY`.
Alternative flow	If the file type is not an accepted video format or the size exceeds the limit, the server rejects the request before issuing a URL, and no orphan record is created.
Postcondition	The video appears on the user's channel, initially as processing and subsequently as ready to play.

Chapter 4

System Design

4.1 High-Level Architecture

The system is organised into four tiers, shown in Figure 4.1: a client tier running in the browser, a delivery tier built on Amazon CloudFront, an application tier comprising the backend API and the database, and a processing tier that performs transcoding.

The separation between the application tier and the processing tier is the defining characteristic of this design. The backend API never touches video content: it issues pre-signed URLs, records metadata, and answers queries, while every byte of video flows directly between the browser, Amazon S3, and CloudFront. This separation has three consequences that together justify the architecture. The API server's resource consumption becomes independent of video size and upload concurrency. The processing tier can scale according to transcoding demand without affecting the responsiveness of the interface. And a failure in the processing tier degrades the system gracefully, leaving already-published videos playable while newly uploaded ones remain in a pending state.

4.2 Transcoding Pipeline Design

Figure 4.2 traces the sequence of interactions from the moment the user initiates an upload until the video becomes available.

Two design decisions embedded in this sequence merit explanation.

The first concerns the ordering of database record creation and URL issuance. The backend creates the database record *before* generating the pre-signed URL, and derives the S3 object key from the identifier of that record. The alternative, generating a key first and creating the record afterwards, introduces a race condition: the S3 event notification can arrive and trigger transcoding before the record exists, at which point the transcoder

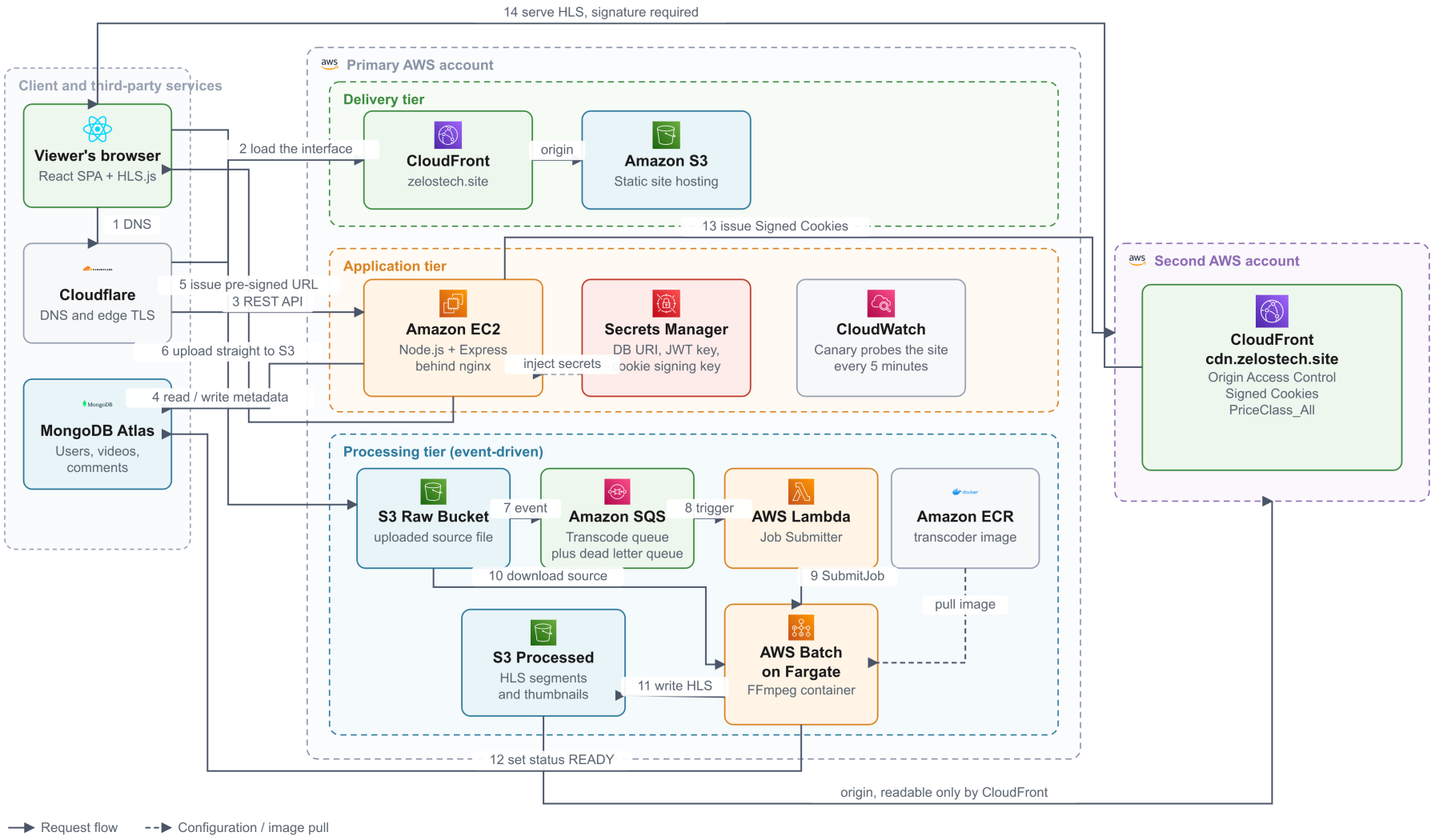


Figure 4.1: *Overall system architecture, showing the deployment boundaries and the numbered data flow from an anonymous visit through upload, transcoding and signed delivery*

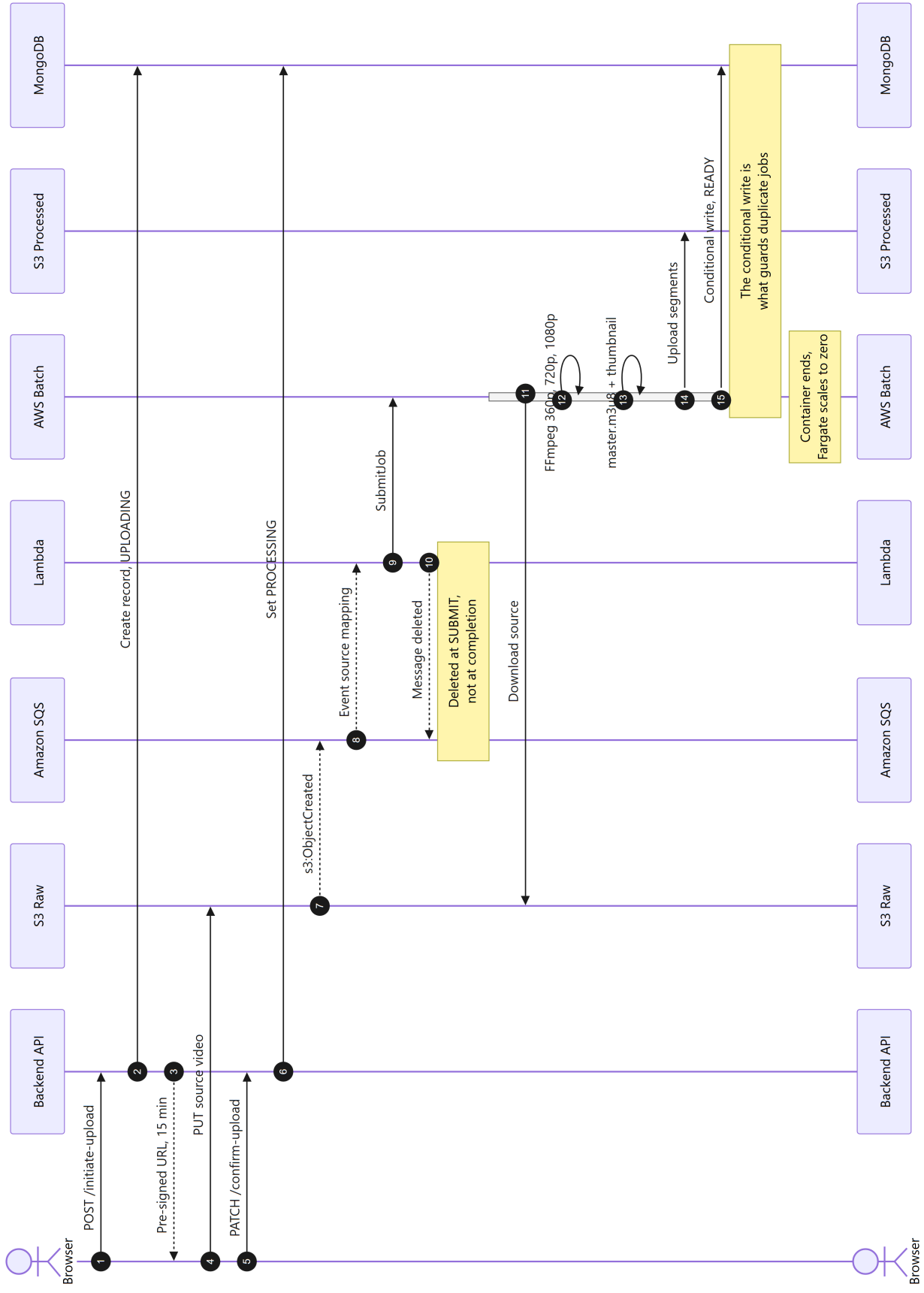


Figure 4.2: *Sequence diagram of the event-driven transcoding pipeline*

has no row to update and the job fails. Creating the record first eliminates this class of failure entirely.

The second concerns the visibility timeout of the SQS message. Amazon SQS hides a message from other consumers for a configured period after it is received, and redelivers it if the consumer has not deleted it by then. Transcoding a large file can easily exceed any reasonable fixed timeout, which would cause the same video to be processed twice concurrently. The transcoder therefore implements the heartbeat pattern: while FFmpeg runs, a timer periodically extends the visibility timeout of the in-flight message. The message is deleted only after the job completes successfully. If the container crashes, the heartbeat stops, the timeout lapses, and the message returns to the queue for another attempt.

That description is accurate about the transcoder's `worker` mode, in which the container consumes from the queue itself, but it is important to be precise that this is not the mode the deployed system runs. The container image and the AWS Batch job definition both invoke `batch` mode, in which the video identifier and object key arrive as environment variables and the container never contacts SQS at all. The queue is consumed instead by the Lambda submitter, which reads the message, submits a Batch job, and returns immediately. Because the Lambda's work is finished at that point, the message is deleted when the job is *submitted*, not when transcoding completes.

The consequence is that on the deployed path there is no heartbeat, and the message is long gone before FFmpeg starts. Two properties the pattern was chosen to provide therefore have to be obtained elsewhere. Protection against duplicate processing rests entirely on the conditional database writes described in Section 5.2, which is where it belongs in any case, since Burrows' analysis of leases (Section 5.6) shows that a renewal timer cannot by itself guarantee mutual exclusion. Retrying a failed job, which the heartbeat design delegated to SQS redelivery, has no equivalent unless AWS Batch is configured to provide it; that it originally was not is discussed in Section 5.5.

Retaining the worker-mode implementation is a deliberate choice rather than an oversight: it allows the transcoder to be run against the queue directly during development without provisioning Batch. Recording which of the two modes the evaluated system actually executes matters, though, because a mechanism present in the source but absent from the deployed path cannot be credited with the guarantees it would provide if it ran. Section 5.5 draws the same distinction about content protection.

4.3 Database Design

The persistence layer uses MongoDB Atlas with three collections, shown in Figure 4.3.

The `Video` document carries a `status` field that acts as a state machine with four values. A record begins as `UPLOADING` when the pre-signed URL is issued, moves to `PROCESSING` once the upload is confirmed and the job is submitted, and finally reaches either `READY` or `ERROR`. Modelling the lifecycle explicitly rather than inferring it from the presence of other fields makes the interface straightforward: the client simply renders a different state for each value, and a video that failed to transcode is distinguishable from one that is merely slow.

Likes and dislikes are stored as arrays of user identifiers embedded within the video document rather than as a separate collection. The denormalisation is justified at this scale because the counts are always read together with the video, and because a single user can appear at most once in each array, so the arrays cannot grow through repeated interaction by the same person.

It should not be justified more strongly than that. Uniqueness bounds each array by the number of distinct users, which is a bound only in the formal sense; on a widely viewed video it is exactly the growth that a document store is poorly suited to hold, against MongoDB's document size limit. The cost is also paid on reads that do not want it, since a listing of twelve videos retrieves every identifier in all twenty-four arrays merely to display counts. The conventional remedy is a separate collection with a compound unique index on video and user, plus a denormalised counter on the video. That is the right structure for a system expecting popular content, and the reason it was not adopted here is scale rather than principle. That is a different claim from the design being correct in general, and it is recorded as such.

Indexes are defined to match the actual query patterns. A compound index on visibility, status, and creation time supports the home page listing, which always filters on the first two fields and sorts on the third.

Search is the exception. The collection carried a text index on title and description, described as supporting the search feature, but no query ever used it: search is implemented as a case-insensitive regular expression matched against title, description and tags, and MongoDB cannot serve an unanchored regular expression from an index. Every search was a full collection scan, and the index cost write throughput and storage while serving nothing. The index has been removed rather than the query rewritten, because switching to a text search would change the feature's behaviour and not merely its cost: the regular expression matches substrings inside words and covers the tags field,

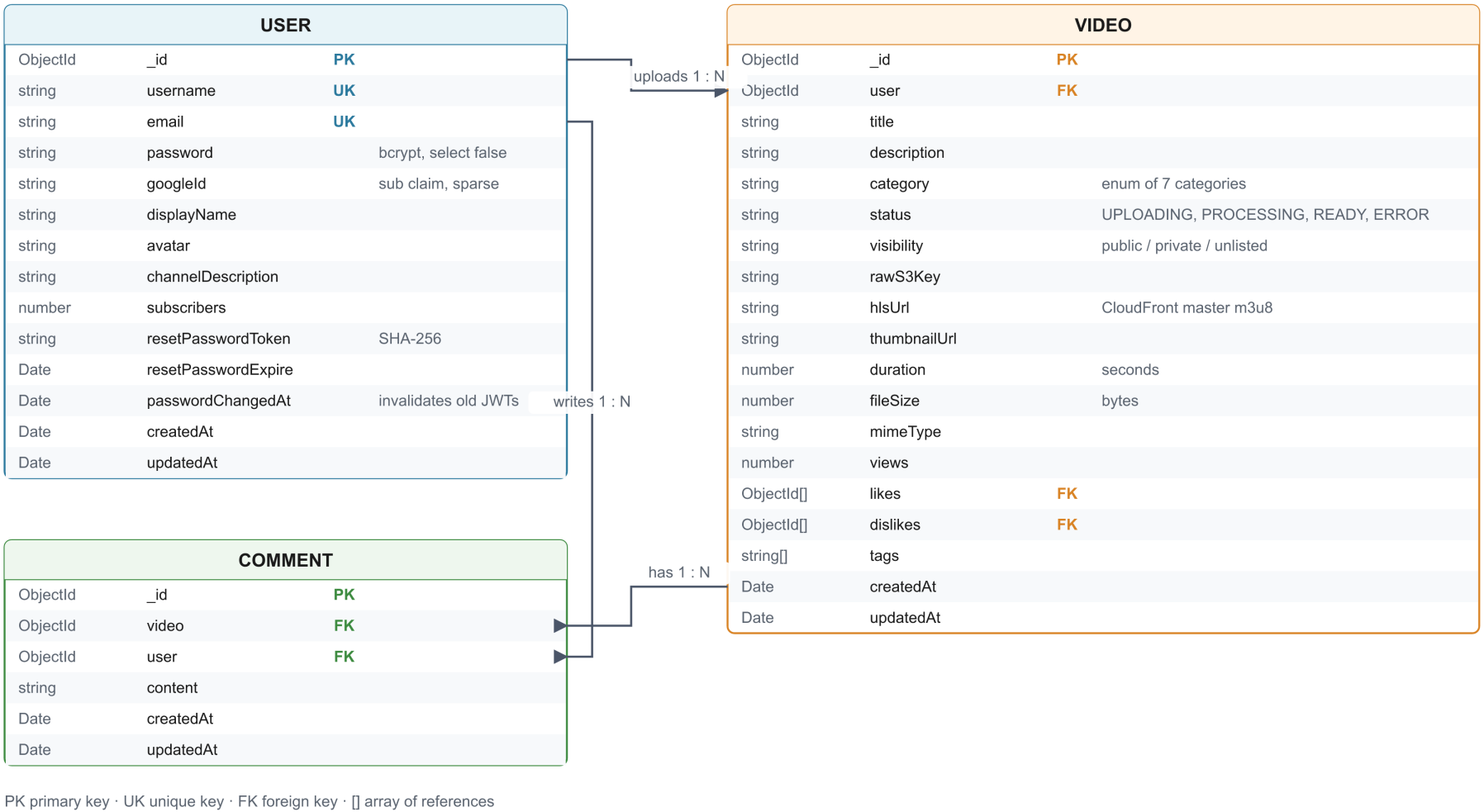


Figure 4.3: *Entity relationship diagram of the MongoDB data model*

neither of which a text index does. At the scale of this project a full scan is acceptable.

4.4 Content Protection Design

Protecting private content requires two complementary mechanisms, because each addresses a different threat.

Origin Access Control makes the processed S3 bucket entirely private and grants read access exclusively to the CloudFront distribution through a bucket policy. This prevents a viewer from bypassing the content delivery network and fetching objects directly from S3, which would both circumvent any access control implemented at the edge and incur higher data transfer charges.

Origin Access Control alone, however, does not determine *who* may retrieve content through CloudFront. Without a second mechanism, any person holding a manifest URL can download the video regardless of its visibility setting, which would make the private mode ineffective in practice. The system therefore adds CloudFront Signed Cookies backed by a trusted key group [8]. The backend verifies the requester's permission using exactly the same service function that governs the API response, signs a policy scoped to the directory of that single video, and sets the resulting cookies on the response. CloudFront rejects any request lacking a valid signature with HTTP 403 at the edge.

Signed cookies were chosen over signed URLs because of the structure of HLS. A playback session retrieves a manifest and then hundreds of individual segment files, each as a separate request. Signing each URL individually would require rewriting every entry inside the manifest, typically using an edge function, which adds considerable complexity and cost. A single set of cookies covers the entire directory and is attached automatically by the browser to every subsequent request.

The policy is signed as a custom policy rather than a canned policy. This distinction is not cosmetic: a canned policy does not permit wildcards in the resource path, whereas the required scope is an entire directory. The custom policy expresses the resource as a pattern covering the manifest and all segments of one specific video, which grants exactly the access needed and no more.

One consequence of this design deserves stating precisely, because it is easy to describe the scheme as protecting private videos and thereby understate it. The edge does not distinguish one visibility level from another: *every* manifest and segment requires a valid signature, and a request without one is refused whatever the video's visibility. What visibility governs is the authorisation decision made earlier, when the backend decides whether to issue cookies at all. All three levels are thus protected

identically at the edge and differ only in who can obtain the credential.

That uniformity has a limit that the design has to accommodate. A cookie authorises exactly one video's directory, whereas a catalogue page renders thumbnails for many videos at once, and no single cookie can cover them. Rather than weaken the policy scope to span the whole bucket, the distribution carves out a separate cache behaviour for the thumbnail path that requires no signature. The justification is that a thumbnail is not the asset the mechanism exists to protect: it is a single low-resolution still, already shown to anyone permitted to see the video in a listing, and treating it as public costs nothing that the scheme was designed to prevent. The narrower point is that the protected unit is the stream, not every byte associated with a video.

4.5 Interface Design

Figure 4.4 sets out the paths a user can take through the application, and it is worth reading before the component-level discussion that follows, because two of its properties constrain the design.

The first is that the catalogue and the watch page are reachable without an account. Browsing, searching, filtering by category and playing a public video all work for an anonymous visitor, and authentication is demanded only at the point where an action requires an identity, which in practice means uploading, commenting, reacting, or managing one's own channel. A visitor who reaches the upload page without a session is therefore diverted through sign-in or registration and returned to it, rather than being asked to sign in on arrival.

The second is that the upload path does not end where the user's involvement ends. Once the browser has transferred the file directly to Amazon S3, the interface returns control immediately and the video enters the `PROCESSING` state, in which it is visible only to its owner on the channel management view. Only when the pipeline marks it `READY` does it join the public catalogue. The interface consequently has to represent a video that exists but cannot yet be played, which is why the state machine of Section 4.3 is modelled explicitly rather than inferred.

The frontend is a single page application composed of functional React components with hooks, using React Context for authentication and theme state. Redux was considered unnecessary at this scale, since the shared state consists only of the current user and the selected theme.

The interface is available in Vietnamese and English, selected by a control in the header and remembered in the browser. One consequence of adding a second language

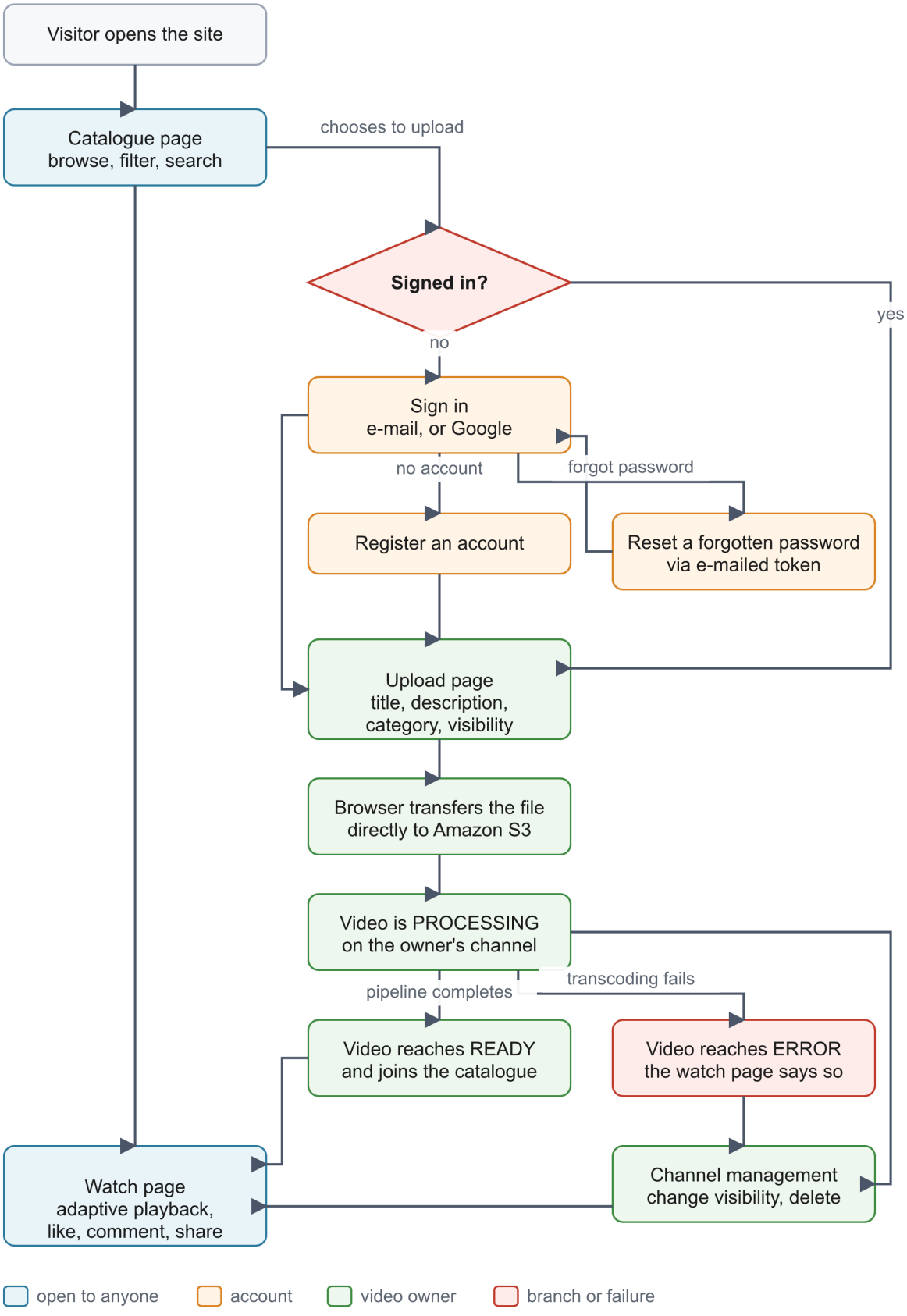


Figure 4.4: *User flow through the application, from an anonymous visit to a published video*

is worth recording, because it constrains the data model rather than only the presentation layer. Video categories are stored in the database as Vietnamese strings, and the schema validates each new video against exactly those strings, so translating a category would have made both the catalogue filter and the upload form reject valid input. The display label and the stored value are therefore kept apart: the client renders a translated label while always sending the original value to the API. The same separation applies to messages the API returns, which are selected from the request's `Accept-Language` header rather than being fixed at the point where the error is raised.

The video player is implemented as a custom component rather than relying on the browser's native controls. The reason is that native controls cannot be extended: the browser renders its own separate menu for playback speed and picture-in-picture, which cannot be merged with the quality selector that HLS.js exposes. Presenting the user with two unrelated menus would be confusing, so the component reimplements play and pause, seeking, volume, fullscreen, playback speed, picture-in-picture, and rendition selection within a single settings menu.

The player also determines when a view is counted. Rather than incrementing the counter when the page loads, which would allow the count to be inflated by repeated refreshes, the component notifies the page only after playback has passed a threshold of several seconds, and the server additionally ignores repeated reports from the same viewer within a thirty-minute window and never counts the owner's own views.

Chapter 5

Development and Testing

5.1 Backend Implementation

The backend follows a layered structure in which routes delegate to controllers, controllers delegate to services, and only services interact with the data models. Business logic is kept out of route handlers so that it can be unit tested without an HTTP server.

Authorisation for video access is concentrated in a single service function. Every path that exposes a video, whether the detail endpoint, the like and comment endpoints, or the playback authorisation endpoint, calls that same function. This is a deliberate choice: duplicating the permission check across endpoints is the most common way for an access control gap to appear, because a later change updates one copy and not the others.

The upload flow validates metadata on the server before issuing a pre-signed URL. Client-side validation alone is insufficient, since a user can call the API directly and obtain a URL for an arbitrary file, effectively turning the project's storage bucket into free hosting for unrelated content. The server therefore checks the declared MIME type against an allow-list of video formats and rejects files exceeding two gigabytes.

Several protective measures were added at the HTTP layer. Security headers are applied through Helmet. Cross-origin requests are restricted to an explicit allow-list rather than being accepted from any origin, which is also a prerequisite for signed cookies to work, because the CORS specification forbids credentials in combination with a wildcard origin. Authentication endpoints are rate limited to ten attempts per fifteen minutes per address, which makes password guessing impractical without affecting legitimate users. Finally, the process validates its required environment variables at startup and refuses to start if the token signing secret is missing, converting a dangerous runtime failure into an immediate and obvious one.

5.2 Transcoder Implementation

The transcoder is a Node.js worker packaged with FFmpeg in a Docker image built through a multi-stage build. The base image went through two upgrades over the course of the project. Node 18 was used initially but had to be abandoned because Mongoose 9 depends on the Web Crypto API, which is not fully available in that version, causing the container to fail when connecting to MongoDB Atlas. Node 20 was then adopted, and later replaced with `node:24-slim` once Node 20 reached end-of-life on 30 April 2026 and stopped receiving security patches; Node 24 was chosen over the newer Node 26 because it is the current Active LTS release, supported until April 2028, whereas Node 26 had not yet entered its LTS phase at the time of the upgrade. The Alpine variant was avoided throughout because FFmpeg requires glibc.

The heartbeat mechanism described in Section 4.2 is, in effect, a lease in the sense introduced by Burrows for the Chubby lock service [9]: a holder retains a resource by periodically renewing it before a timeout expires, and Burrows already identified the classical failure mode of that pattern: a lease can expire while its holder is merely stalled rather than dead, so a second holder may be granted the same resource while the first is still alive and will eventually act on it. A defect matching exactly this failure mode surfaced during a literature-driven review of the pipeline's reliability guarantees, discussed further in Section 5.6: neither `updateVideoReady` nor `updateVideoError` checked the video's current status before writing, so a duplicate job triggered by an SQS redelivery could silently overwrite a successful result, and in the worst case could revert an already-`READY` video back to `ERROR`. Both functions were changed to a conditional `findOneAndUpdate` that only writes when the current status is not already `READY`, and `processVideo` now checks the video's status before downloading anything, so a duplicate delivery that arrives after the first job has already finished is rejected immediately instead of re-downloading and re-transcoding the source.

A single FFmpeg invocation produces all three renditions, which is more efficient than three separate invocations because the source is decoded only once. The command emits segments of six seconds, generates the per-rendition playlists, and the worker then writes the master playlist that lists the available bandwidths, codecs and resolutions. Those bandwidths and codec strings are derived from the encoded output rather than from the encoder settings: the bandwidth is the peak segment bit rate measured from the segments themselves, which Section 6.3.3 examines in detail, and the codec string is read out of each stream's sequence parameter set, since the constraint-set byte that RFC 6381 requires appears in neither the profile name nor the level number reported by a probe. A

thumbnail is extracted in a separate pass.

Failure handling is deliberate. When FFmpeg exits with a non-zero status, the error propagates to the top-level handler, which records the `ERROR` status in the database and, in `worker` mode, does *not* delete the SQS message: leaving it in flight allows the visibility timeout to lapse so that the job is retried, and after the configured number of failed attempts the message is moved to the dead letter queue where it can be inspected rather than lost.

As Section 4.2 establishes, however, the deployed system runs `batch` mode, where the container holds no message to retain. The retry and dead-letter behaviour just described therefore governs only the Lambda that submits jobs, not the transcoding itself, and the recording of `ERROR` in the database is the sole part of this paragraph that applies to a deployed transcoding failure. The gap this left, and its consequences for how such failures were detected, is examined in Section 5.5.

5.3 Infrastructure as Code

The AWS environment is described by eleven Terraform modules, summarised in Table 5.1, and is instantiated separately for the development and production environments.

Table 5.1: Terraform modules and their responsibilities

MODULE	RESPONSIBILITY
`s3`	Raw and processed buckets, CORS configuration, Glacier lifecycle transition, event notification
`sqs`	Transcode queue, dead letter queue with a redrive policy, queue access policy
`ecr`	Container registry with image scanning on push and a lifecycle policy retaining recent images
`iam`	Execution roles for AWS Batch, the ECS task, the transcoder task, and the Lambda submitter
`vpc`	Virtual network, public subnets, internet gateway, security groups, S3 gateway endpoint
`secrets`	AWS Secrets Manager entries for the database connection string and the token signing secret
`sns`	Notification topics for transcode completion and dead letter queue alerts
`monitoring`	CloudWatch log groups, alarms, and dashboard
`batch`	Fargate compute environment, job queue, and job definition
`lambda`	Job submitter function and its SQS event source mapping
`cloudfront`	Distribution, Origin Access Control, cache policy, and the trusted key group used for signed cookies

Two configuration details are worth highlighting. The S3 lifecycle policy transitions

raw source files to Glacier after thirty days, since they are never read again once transcoding has succeeded but must be retained in case a re-encode becomes necessary. The cache policy assigns a long time-to-live to segments, which are immutable once produced, and a short one to manifests, which may change if the set of renditions changes.

5.4 CI/CD Pipeline and DevSecOps

Seven GitHub Actions workflows automate verification and deployment, as shown in Figure 5.1.

The security gate is structured in two tiers, and the reason is practical. Scanning at the combined critical and high severity levels and failing the build on any finding sounds rigorous, but in practice most high-severity findings originate in transitive dependencies for which no patch yet exists. A gate configured that way fails continuously for reasons the team cannot act upon, and the predictable result is that developers begin ignoring it. The pipeline therefore reports critical and high findings for visibility without blocking, and blocks only on critical findings for which a fix is available. This keeps the gate meaningful, because a failure now always corresponds to an action the team can actually take.

Static application security testing is performed with `eslint-plugin-security` rather than SonarQube. The rule set detects patterns such as command execution built from user input, regular expressions vulnerable to catastrophic backtracking, and cryptographically unsafe random number generation. The choice avoids operating a separate analysis server while still executing genuinely within the pipeline. Where a finding is a false positive, it is suppressed with an inline annotation that documents why, which forces each suppression to be justified rather than silently ignored.

Infrastructure code is validated in its own workflow, which checks formatting, initialises without a remote backend so that no credentials are required, validates both environments, and scans the configuration for insecure settings.

A later audit of the pipeline found a weakness in its own supply chain. All five workflows referenced the vulnerability scanner as `aquasecurity/trivy-action@master`. A branch reference resolves to whatever that repository's default branch contains at the moment a job runs, so a change made there, including one made by someone who had compromised the action, would execute inside this project's continuous integration, which holds AWS credentials. Every other action in the pipeline was already pinned at least to a major version tag. The references are now pinned to a release commit hash, with the version tag retained in a comment so that upgrades remain deliberate and readable.

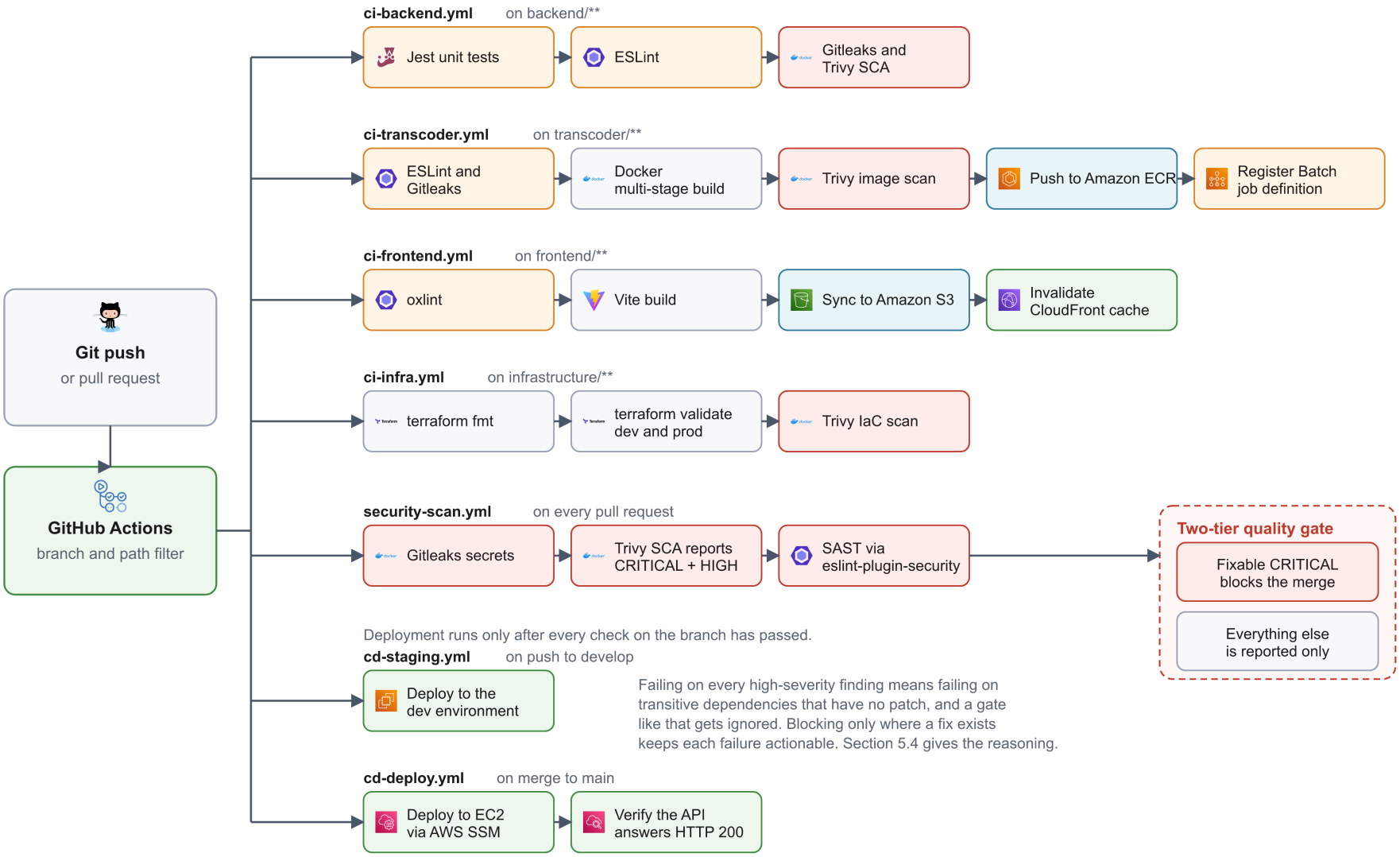


Figure 5.1: *The seven GitHub Actions workflows, the file paths that trigger each of them, and the two-tier DevSecOps quality gate*

The same audit narrowed one permission. The Lambda that submits transcoding jobs held `batch:SubmitJob` on `Resource: "*"`, which allowed it to submit to any job queue in the account rather than the project's own. It is now scoped to the specific queue and job definition. The scoping is expressed through the naming convention rather than by referencing the Batch module's outputs, because the Batch module already depends on the IAM module for its roles and referring back would introduce a dependency cycle; both modules carry a note recording the coupling that this creates. The companion action `batch:DescribeJobs` keeps the wildcard, because it accepts arbitrary job identifiers and AWS provides no resource-level control for it, and the module records that reason in a comment so the wildcard is not mistaken for an oversight.

5.5 Challenges During Development

Several difficulties arose between a design that was correct on paper and a system that behaved correctly once deployed. Each is recorded here with the resolution adopted, because those resolutions shaped parts of the final architecture.

5.5.1 Provisioning Blocked by Account Policy

The content protection scheme of Section 4.4 was implemented and covered by unit tests well before it could be provisioned. Applying the Terraform configuration failed with an access-denied error, because AWS requires manual account verification before a new account may create CloudFront distributions, and the request filed to lift that restriction remained pending with no bound on how long it might take. Until it was resolved the processed-content bucket was reachable directly over the public internet, so the private visibility setting had no effect at the storage layer.

The restriction was worked around by provisioning the distribution in a second AWS account not subject to it. Terraform supports this without duplicating any module: the distribution, its Origin Access Control, its cache and response-header policies, and its Signed Cookie trusted key group were given an explicit `provider` argument pointing at the second account, while the buckets, Batch, Lambda and the database connection remained in the original one. Only the bucket policy could not move, since a bucket policy may be attached only by the account that owns the bucket. This required no special handling, because an Origin Access Control policy is expressed as a service principal together with an `AWS:SourceArn` condition naming the distribution, a form that is indifferent to which account owns the ARN. With the distribution in place, the

temporary public bucket policy was removed and the bucket returned to being reachable only through CloudFront.

5.5.2 A Signed Cookie Policy That Matched Nothing

Exercising Signed Cookies against the new distribution revealed that every request returned `403 Forbidden`, whatever the video's visibility. The function building the resource pattern that a cookie authorises, `buildResourcePattern`, signed `videos/*/{videoId}/*`, with a wildcard segment between the prefix and the identifier, whereas the transcoder writes every rendition and the master playlist directly under `videos/{videoId}/`. Each cookie was therefore well formed and correctly signed, and authorised a resource that no viewer could request.

The existing unit test did not detect this because it compared the function's output against a literal string copied from the function itself rather than against the transcoder's key layout, so it could confirm only that the function had not changed. The pattern was corrected to `videos/{videoId}/*` and the test rewritten to assert against the layout the transcoder actually produces. Signed Cookies were then verified end to end against the deployed system: a manifest request without a valid cookie is rejected at the edge, and the identical request succeeds once the playback-authorisation endpoint has issued cookies for that video.

5.5.3 Two Consequences of Gating Every Object

Gating the whole bucket behind Signed Cookies also gated video thumbnails, which a catalogue listing must render for many videos at once even though a cookie authorises exactly one. A separate unsigned cache behaviour was carved out for the thumbnail path, since a thumbnail is not the content the mechanism exists to protect.

Separately, the frontend's own CloudFront distribution, added because an Amazon S3 website endpoint cannot return `200` for a single-page application's client-side routes on direct navigation, had no cache-invalidation step in its deployment workflow, so a fix already synced to the origin bucket continued to be served from the edge for hours afterwards. The workflow now invalidates that distribution immediately after every deployment.

5.5.4 Playback Across the Cookie Expiry Boundary

Signed Cookies are valid for two hours, and the watch page requested them once when it mounted. Crossing that boundary produced a `403` on the next segment, and the player's error handler responded to any fatal network error by calling `startLoad()` with no attempt counter and no delay, so an error that does not resolve on its own became an unbounded retry loop behind a "reconnecting" message that never changed.

Two properties of hls.js governed the fix. The library marks the first `403` as non-fatal while it retries internally, so the handler must inspect the status code before filtering on the fatal flag, or it discards the very error it exists to report. And `startLoad()` resumes segment loading, whereas an expired cookie fails at the manifest, so recovery requires re-invoking `loadSource()`. The handler now refreshes the cookies on a `403`, reloads the source, and gives up after a bounded number of attempts with a message the viewer can act on.

5.5.5 An Empty Secret That Stopped the Pipeline

Every transcoding task began failing at `ResourceInitializationError` before its entrypoint ran. The Terraform module for Secrets Manager created the entry holding the notification e-mail password unconditionally but populated its value only when a password had been configured, so leaving e-mail unconfigured produced a secret with no version carrying the `AWSCURRENT` label. The Batch module decided whether to declare that secret in the job definition by testing whether its ARN was non-empty, and the ARN of an empty secret is non-empty. The fix made the secret itself conditional, so that the unconfigured state is internally consistent: no secret, an empty ARN, and no declaration in the job definition.

The defect persisted longer than it should have because the only jobs submitted while it was live belonged to the concurrency stress test, whose synthetic payloads were expected to fail. Checking the cause recorded in `statusReason` rather than the terminal state alone would have identified it immediately, and the alerting described next was changed to carry that field.

5.5.6 Retry and Alerting on the Deployed Path

Section 4.2 describes retry as delegated to SQS redelivery, which holds in `worker` mode. On the deployed path the Lambda submitter deletes the message once the job is submitted, so SQS no longer holds anything to redeliver, and the Batch job definition declared no

`retryStrategy`. Nothing retried a failed job at any layer, and a transient fault such as a network error while downloading the source lost the video with no record beyond an `ERROR` row.

Detection had the same shape. The alarm whose description read "video transcoding failures detected" watched the depth of the dead letter queue, which receives a message only when the Lambda fails to submit, so a job that fails after submission is invisible to it. The job definition now declares three attempts, retrying infrastructure faults and exiting immediately when the exit status indicates that FFmpeg rejected the content, together with a two-hour timeout that bounds a process stalled on malformed input. Because AWS Batch publishes no CloudWatch metric for failed jobs, an EventBridge rule matches job state transitions to `FAILED` and publishes to the alert topic, with the notification carrying `statusReason`. The alarm description was corrected to state what it genuinely covers.

5.5.7 Monitoring the Service Rather Than the Server

Both of the alarms defined at that point watched the transcoding pipeline, the dead letter queue and the queue depth, and neither watched the backend API, which runs as a single process on a single instance. The gap surfaced when assigning the backend a static address moved its public IP while DNS still pointed at the released one, and the API returned `522` for several minutes. Every infrastructure indicator stayed green, because nothing at the infrastructure level was wrong.

A scheduled function now calls the public domain every five minutes along the same path a viewer takes and publishes a metric that an alarm watches, with an instance status-check alarm alongside it. A self-hosted check was chosen over a managed one on cost, since a managed health check on an endpoint outside the provider costs roughly 1.75 USD per month against zero within the free tier, and the trade-off it accepts, that checks originate from a single location, is recorded with the implementation. The alarm's evaluation period is three times the check interval, so that each window holds several datapoints and one late execution cannot open a gap that the missing-data policy would read as a breach.

5.5.8 Infrastructure Drift in the Instance Definition

The AMI for the backend instance is resolved through a data source with `most_recent = true`, and the `ami` attribute forces replacement when it changes, so every Ubuntu security release armed the next `terraform apply` to destroy and rebuild the running server. What would have been lost is the state Terraform does not manage: the environment file

holding the database URI, signing key and mail credentials, the nginx configuration, the TLS certificates, the process manager's state, and the public address that DNS points at. Changes to the attribute are now ignored, which makes an image upgrade deliberate, and an Elastic IP keeps the address stable across any rebuild that does occur.

5.6 Testing

The automated test suite comprises 209 tests across fourteen suites, summarised in Table 5.2.

Table 5.2: Automated test coverage by area

SUITE	TESTS	FOCUS
videoService	11	Upload initiation, ownership checks, deletion cascade
s3Service	2	Key generation and pre-signed URL creation
authService	17	Registration conflicts, login failures, token claims, password reset
authMiddleware	14	Missing, malformed, forged, and expired tokens, and tokens issued before the account's last password change
visibility	21	Privacy enforcement across every query path, view deduplication
uploadValidation	11	HTTP-level validation of file type and size
cloudfrontService	24	Cookie signing, policy scope, cookie attributes, and that revocation repeats the attributes used when issuing
forgotPassword	5	That the response is identical whether or not the address has an account, including when e-mail delivery fails
relatedVideos	10	That the recommendation endpoint reuses the same authorisation rule as the detail endpoint, and never surfaces a private or unlisted video
transcoder/dbHandler	6	Conditional writes preventing duplicate-job overwrites
transcoder/emailService	6	READY/ERROR notification content and recipient targeting, and that raw failure detail is never exposed to the viewer
transcoder/framerate	32	That the group of pictures follows the real source frame rate, that a variable-frame-rate file cannot poison it, and that every rendition is forced to cut at the same instant
transcoder/bandwidth	18	That the declared bandwidth of each variant is the peak measured from the segments themselves rather than a configured target, rounded upward so it remains the upper bound RFC 8216 requires
transcoder/codecs	32	That the codec string declared for each variant is read from the encoded stream itself rather than derived from configuration, and is omitted rather than guessed when it cannot be read

The visibility suite is the most important from a security standpoint, since it asserts that a private video is absent from the public listing, returns a not-found response rather than a forbidden one to non-owners, and remains accessible to its owner. Returning not-found is deliberate: a forbidden response would confirm that a video with that identifier exists, which leaks information.

The value of testing was demonstrated concretely during development of the signing service. An initial implementation produced a canned policy, which the test suite immediately flagged because the resulting cookie set contained an expiry field instead of a policy field. Since canned policies do not support wildcard resources, that implementation would have failed for every video once deployed. The defect was caught before any infrastructure was provisioned.

Testing caught a second defect in a similar way, this time surfaced not by writing a test first but by a literature review of the reliability guarantees the pipeline was implicitly assuming (Section 5.2). Once the conditional-write fix was implemented, six tests were written to pin down the exact contract: a normal write succeeds when the video is not yet `READY`; a write that loses the race to an earlier successful write is rejected and the function returns the winning record rather than throwing; and a late failure from a duplicate job can never revert an already-`READY` video to `ERROR`. As a falsifiability check, the same six tests were run against the original, unconditional implementation: four of the six failed, either because it never calls `findOneAndUpdate` at all or because it crashes outright when handed a plain record with no `save` method, which is exactly the shape a conditional read would have to return. The two that still passed were the not-found cases, which neither version distinguishes from a duplicate write. The result confirms both that the new tests are actually exercising the fixed code path and that the original implementation could not have satisfied the contract by construction, not merely by accident of the test data chosen.

Chapter 6

Deployment and Evaluation

6.1 The Deployed Environment

The system described in the preceding chapters runs as a live deployment, and every figure reported later in this chapter was measured against it rather than against a local approximation.

The application is reached at two hostnames. The single-page frontend is served from an Amazon S3 bucket through its own CloudFront distribution, and the backend API answers on a separate subdomain from a Node.js process on an Amazon EC2 instance, behind nginx for TLS termination and behind a process manager that restarts it on failure. Media is served from a third hostname belonging to the video distribution, whose origin is the processed-content bucket and which admits a request only when it carries a valid Signed Cookie. Cloudflare provides DNS and edge TLS in front of the two application hostnames. MongoDB Atlas holds the database, and the transcoding tier consists of the SQS queue, the Lambda submitter and the AWS Batch compute environment described in Chapter 5, none of which occupies any capacity between jobs.

A scheduled health check calls the public API domain every five minutes and publishes a metric that an alarm watches, so that the availability of the service, rather than of the instance hosting it, is what is monitored. Because the transcoding tier is billed only while a job runs, the instance is the only component that accrues cost continuously, and it can be stopped and restarted without losing the environment file, the TLS certificates or the public address, all of which survive by design.

6.2 User Interface Showcase

This section presents the interface as it appears to a user of the deployed system. The screens below are described in terms of what a user can do on each and how one leads to the next, rather than merely labelled.

Figure 6.1 shows the catalogue that a visitor reaches first. The header carries the search field, a theme toggle, and the sign-in and registration links, and directly beneath it a row of seven category tabs filters the listing without a page reload. Each card in the grid carries the thumbnail generated during transcoding, a duration badge in its corner, the title, the uploading channel with its avatar, the view count and the age of the upload. A visitor who is not signed in sees the same catalogue as a registered user, since public videos are readable without an account; the difference appears only when an action requires ownership or identity.

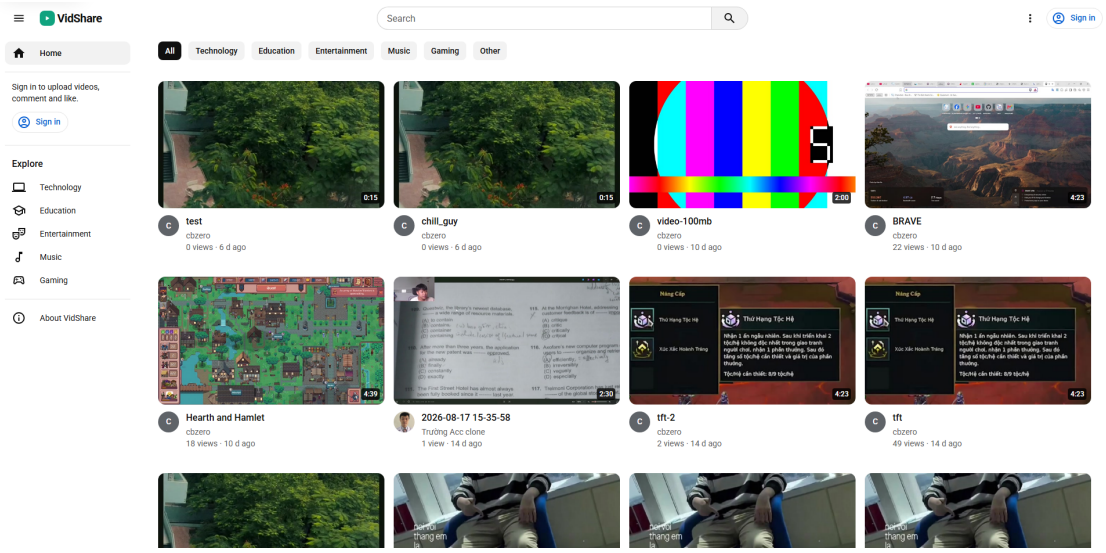


Figure 6.1: *Catalogue page, the entry point for a visitor*

Selecting a card opens the watch page in Figure 6.2. The player is a custom component rather than the browser's native controls, for the reason given in Section 4.5: the quality selector that HLS.js exposes cannot be merged into the browser's own menu, so play and pause, seeking with a timestamp tooltip that follows the cursor along the timeline, volume, fullscreen, playback speed, picture-in-picture and rendition selection are all reimplemented behind a custom control bar. The tooltip reports the time under the cursor alongside the video poster frame; it does not show the frame at that position, since the transcoder extracts a single thumbnail rather than the sprite sheet a frame-accurate preview would require. Before the stream can load, the page calls the playback-authorisation endpoint, which returns the Signed Cookies the content delivery network

requires; only then does the player fetch the master playlist. Below the player sit the title, view count and publication date, the like and dislike controls, a share control, the uploading channel, and the comment thread. To the right of the player area, a responsive sidebar displays recommended related videos filtered by category and author.

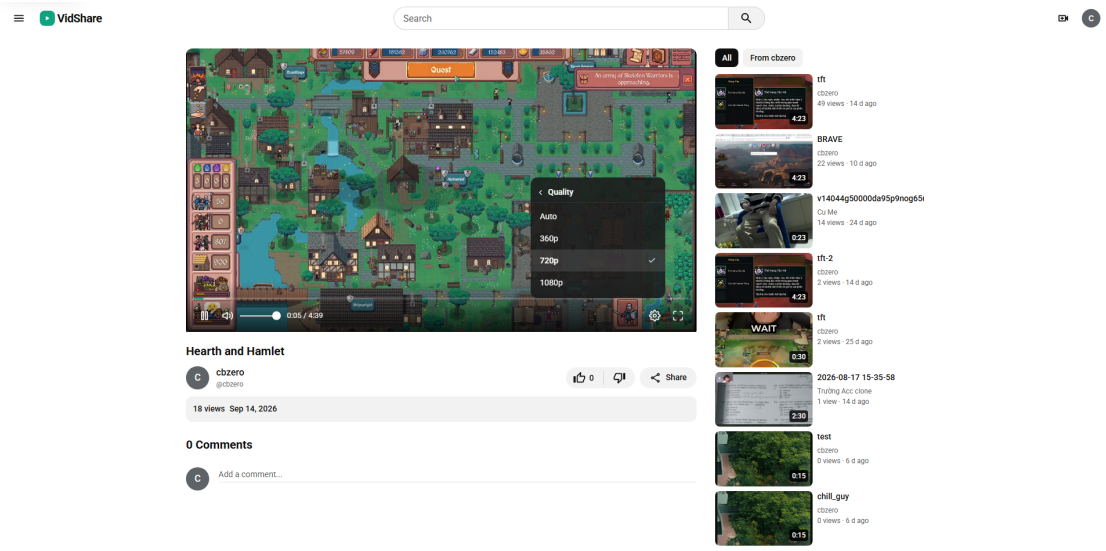


Figure 6.2: *Watch page with custom player, rendition selector and related videos sidebar*

Figure 6.3 shows the upload flow, which is where the architecture of Chapter 4 becomes visible to the user. The form collects the title, description, category and visibility, and on submission the browser receives a pre-signed URL and transfers the file directly to Amazon S3, so the progress indicator reflects a transfer that never passes through the backend. During upload, the interface displays live transfer telemetry including uploaded bytes, instantaneous transfer speed, and estimated time to completion (ETA), while providing a cancel button backed by an `AbortController` to abort the in-flight S3 PUT request immediately and asynchronously discard the draft `UPLOADING` database record to prevent orphaned records from accumulating. Once the transfer completes the page returns control immediately rather than waiting for transcoding, and the video appears on the uploader's channel in a processing state until the pipeline marks it ready.

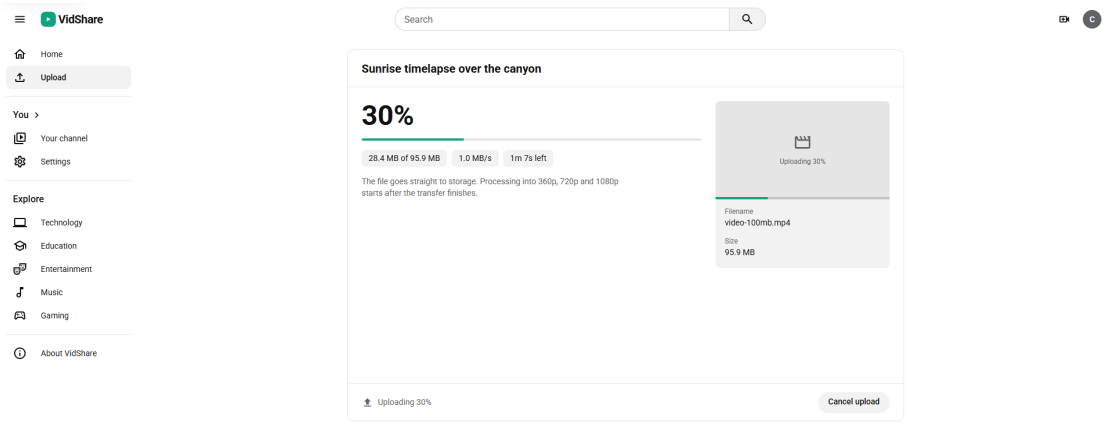


Figure 6.3: *Upload page, showing direct-to-storage transfer with live telemetry*

Figure 6.4 shows the channel management view, which lists the signed-in user's own videos regardless of visibility, together with the state of each. This is the only place a video in the `PROCESSING` or `ERROR` state is visible, since the public catalogue lists only videos that have reached `READY`, and it is where a video has its visibility changed (either directly via the actions menu or through a modal editing form that updates title, description, category, and visibility), or is deleted.

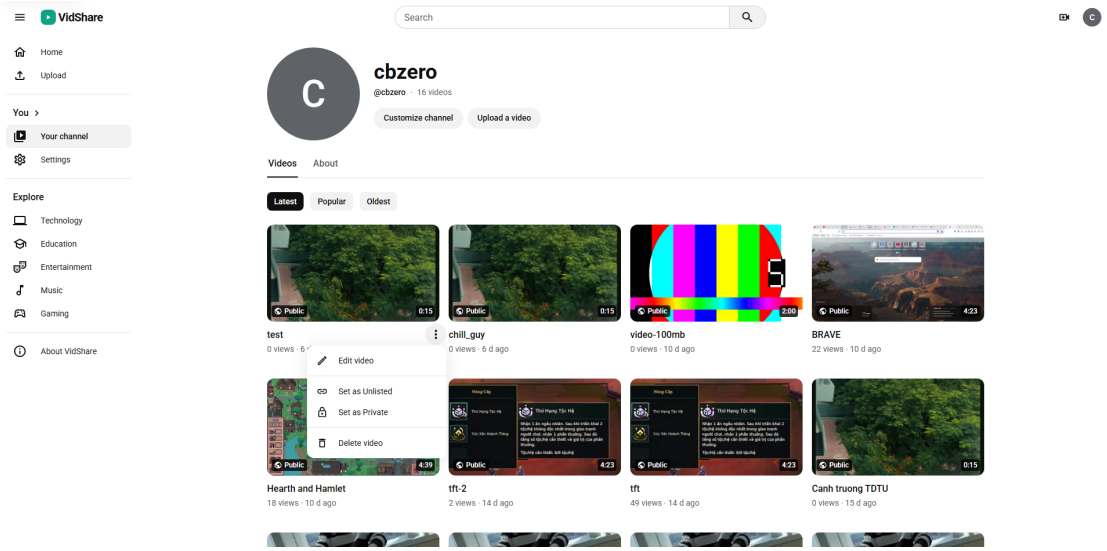


Figure 6.4: *Channel management view, including videos still being processed*

6.3 Quality of Experience Evaluation

Time-to-First-Frame is measured by an instrumentation script that fetches the master playlist, parses it to locate a rendition playlist, fetches that playlist, and finally downloads the first media segment, recording the elapsed time for each stage. The measurement is repeated so that mean, median, and 95th percentile values can be reported, because a single sample is meaningless when network latency varies and the first request is typically an edge cache miss.

Table 6.1: Time-to-First-Frame measurement results

STAGE	MEAN	MEDIAN	P95	MIN	MAX
Master playlist	95.55	82.49	355.02	76.61	355.02
Rendition playlist	88.62	82.40	207.32	77.62	207.32
First segment	108.76	87.04	322.20	78.41	322.20
Total TTFF	**292.93**	**251.52**	**884.54**	**242.23**	**884.54**

Note: values in milliseconds, n = 20 samples, measured against a video served directly from the S3 bucket, before the CloudFront distribution described in Section 5.5 was deployed. These figures therefore represent origin latency without any CDN, and they serve here as the baseline against which Section 6.3.1 compares the deployed system. The mean (292.93 ms) exceeds the median (251.52 ms) because one outlier sample (884.54 ms) inflates both the mean and the P95/max columns, which coincide with the single-sample maximum at this sample size.

6.3.1 Measuring the Deployed CDN, and a Configuration That Inverted It

Repeating the measurement against the live distribution required extending the instrumentation, since every request now requires a Signed Cookie: without one the script would have measured the latency of being refused. The script therefore requests cookies from the playback-authorisation endpoint and attaches them to all three stages, exactly as a browser does.

The first run produced a result that contradicted the entire justification for using a content delivery network. Against the CDN the median TTFF was 727.69 ms, against 251.52 ms measured directly from S3. The CDN was slower than the origin it was meant to accelerate, by a factor of 2.9. Caching was not the problem: nineteen of twenty samples were edge hits, and the single miss (4655 ms) was six times slower than the hits,

so the cache was working precisely as intended.

The cause was visible in a response header rather than in any timing. Every request reported `X-Amz-Cf-Pop: MRS53`, the edge location in Marseille, France, and the frontend distribution was answering from Tel Aviv. Both distributions were configured with `PriceClass_100`, annotated in the Terraform as the cheapest option, which restricts delivery to edge locations in North America and Europe. Every request from Vietnam, where the system's users are, was therefore crossing to Europe and back. The setting did reduce the per-gigabyte rate, and in doing so it removed the entire benefit the component exists to provide for the only population that uses it.

Changing both distributions to `PriceClass_200`, which includes the Asian edge locations, moved delivery to `SGN50` in Ho Chi Minh City and produced Table 6.2.

Table 6.2: Time-to-First-Frame through CloudFront after the price class correction

STAGE	**MEAN**	**MEDIAN**	**P95**	**MIN**	**MAX**
Master playlist	28.65	8.37	263.43	6.54	263.43
Rendition playlist	29.29	7.93	348.78	5.89	348.78
First segment	112.75	87.36	487.31	68.87	487.31
Total TTFF	**170.69**	**110.32**	**1099.51**	**83.17**	**1099.51**
Warm cache only (n = 18)	*108.49*	*107.57*	*159.25*	*83.17*	*159.25*

Note: values in milliseconds, $n = 20$ samples with Signed Cookies attached. Two samples were edge cache misses and eighteen were hits; the final row reports only the samples that hit at all three stages, which is the steady state a viewer of an already-watched video experiences. The P95 and maximum of the full sample coincide with the colder of the two misses, as at this sample size the 95th percentile is the single slowest observation.

The comparison against the two earlier figures is stark. The median falls from 727.69 ms through the European edge to 110.32 ms through the local one, a factor of 6.6, and now sits 2.3 times faster than the S3 origin baseline rather than 2.9 times slower. The stage breakdown shows where the gain lands: the manifest and rendition playlists, small files served from cache, now complete in single-digit milliseconds, while the segment continues to dominate because it is the only stage transferring a meaningful payload.

Two properties of this defect are worth noting for the measurement methodology. It was invisible to every form of inspection short of measurement from the right place: the infrastructure code was valid, the distribution was healthy, the cache hit rate was excellent, and each individual request succeeded, so a measurement taken from Europe or North America would have shown the CDN performing well. And the original setting was a deliberate, documented cost optimisation that was correct about cost; what it did

not account for was the geography of the traffic it governed.

6.3.2 Delivery Latency by Client Location

Since a single vantage point was what concealed the fault above, the measurement was repeated from five. The same three-stage probe, with Signed Cookies attached, was deployed as an AWS Lambda function to five regions, invoked for twenty iterations each, and deleted afterwards. Placing the client inside AWS regions makes the comparison between locations clean, because the only variable that changes is geography; the caveat this introduces is discussed below. Table 6.3 reports the results.

Table 6.3: Time-to-First-Frame by client location, before and after `PriceClass_All`

	PRICECLASS_200		PRICECLASS_ALL		
CLIENT LOCATION	**EDGE**	**MEDIAN**	**EDGE**	**MEDIAN**	**WARM**
N. Virginia, United States	IAD61	55.23	n/a	n/a	n/a
Dublin, Ireland	DUB2	55.91	DUB2	55.68	55.68
Singapore	SIN3	56.89	SIN3	53.20	53.20
Tokyo, Japan	NRT12	59.68	n/a	n/a	n/a
Sydney, Australia	n/a	n/a	SYD3	52.30	43.36
São Paulo, Brazil	CPT51	**1196.10**	GRU1	**36.79**	**36.49**
Ho Chi Minh City (consumer)	SGN50	110.32	n/a	n/a	n/a

Note: values in milliseconds, $n = 20$ per location and per configuration. WARM reports the median over samples that hit the edge cache at all three stages. Entries marked n/a were not measured in that configuration: the first run predated the Sydney probe, and the second re-measured only the two locations that could change plus two controls. The Ho Chi Minh City row is the local measurement of Table 6.2, taken from a consumer connection rather than from inside an AWS region, and is included for comparison rather than as a like-for-like sample.

Under `PriceClass_200`, four of the five locations were indistinguishable, clustering between 55 and 60 ms and each served by an edge in its own region. That uniformity is the property a content delivery network is expected to provide, and it corroborates the price class correction more strongly than the single improved figure from Vietnam had.

São Paulo was the exception. Requests from Brazil were served by `CPT51` in Cape Town, South Africa, across the South Atlantic, at a median of 1196 ms, twenty-one times the latency of every other location and considerably worse than the 728 ms that prompted the correction in the first place. The cause was the same mechanism: `PriceClass_200` includes Asia, India, the Middle East and Africa, but excludes South America and

Oceania, so a Brazilian viewer occupied exactly the position a Vietnamese viewer had occupied before. The correction had not eliminated the problem; it had moved the excluded region to one where this system happens to have no users.

Both distributions were therefore moved to `PriceClass_All`, which excludes nothing, and the affected locations were re-measured. Brazil is now served by `GRU1` in São Paulo itself at a median of 36.79 ms, a thirty-two-fold improvement and the fastest figure in the table; Sydney, the other previously excluded region, is served by `SYD3` at 52.30 ms. The two control locations moved by less than 4 ms, confirming that nothing else changed. Delivery is now uniform across every location measured.

The basis for that decision is narrower than the numbers alone suggest. The system has no users in South America or Oceania, so the change did not meet an existing need. It was adopted because the difference in cost is zero in practice: CloudFront's perpetual free tier covers one terabyte of transfer and ten million requests per month, well above anything this deployment approaches. With no cost to weigh against it, there is no reason to retain a region served an order of magnitude worse. Were traffic ever to exceed that allowance, this is the first setting that should be revisited, since South America and Oceania carry the highest per-gigabyte rates in the schedule.

One methodological limit applies to the whole table. These probes ran inside AWS regions, reaching edges over well-provisioned network paths, which is why four of them report around 55 ms while the consumer connection in Ho Chi Minh City reports 110 ms to an edge in the same city. The figures therefore measure edge proximity under favourable conditions, not what a residential viewer in each location would experience; the last-mile difference is roughly a factor of two on this evidence. Within that limit, the table establishes what a single vantage point could not: delivery is uniform wherever the price class reaches, and an order of magnitude worse where it does not.

6.3.3 Playback Under Constrained Bandwidth, and a Manifest That Understated Itself

The Time-to-First-Frame figures above were produced by a script built on `fetch`, and that construction places three quantities permanently out of its reach. Rebuffering ratio, bitrate switch count, and the bitrate a viewer actually receives do not exist unless something is playing: they require a buffer, a playback clock, and an adaptive bitrate algorithm, none of which a sequence of HTTP requests possesses. Measuring them therefore required driving the real player rather than simulating it.

The harness written for this purpose launches headless Chromium under Playwright,

loads the deployed watch page, and takes its signals from the <video> element itself: the waiting, playing, and ended events for stalls, and the resize event for the rendition currently on screen. No production code was modified to accommodate the measurement. Bandwidth is capped through the Chrome DevTools Protocol, which is why the harness is fixed to Chromium. Three definitional choices matter when reading the numbers. Startup buffering is excluded from the rebuffering ratio, because waiting for a video to begin is an ordinary expectation while a mid-playback freeze is not, and conflating them inflates every figure. The initial rendition selection is not counted as a switch, since counting it would add exactly one spurious switch to every measurement. The average bitrate is weighted by the time each rendition was displayed, because playing 1080p for one second and 360p for fifty-nine and then averaging the two rungs describes nothing a viewer experienced.

Each profile discards two warm-up runs before recording five. One proved insufficient, and this was measured rather than assumed: on the slow 3G profile with a single warm-up, the first recorded run showed a startup delay of 26.04 s against roughly 1.90 s for the remaining four, and a second warm-up removed the outlier entirely. The effect is invisible on a fast connection, so testing only the unthrottled profile would have concealed it.

Table 6.4: Playback quality by emulated network profile

METRIC	UNTHROTTLED	FAST 3G	SLOW 3G
Startup delay, median (s)	2.576	2.076	3.803
Startup delay, max (s)	5.221	2.389	3.920
Rebuffering ratio, median	0.00%	0.00%	5.72%
Rebuffering ratio, mean	0.00%	1.20%	6.49%
Rebuffering ratio, max	0.00%	5.99%	14.83%
Runs exhibiting a stall	0/5	1/5	3/5
Bitrate switches, median	1	4	5
Average bitrate (kbit/s)	1833	1551	1200

Note: five recorded runs per profile after two discarded warm-up runs, 120 s of playback each, against a single 4:23 source. Emulated ceilings are 1678 kbit/s for fast 3G and 512 kbit/s for slow 3G. The rebuffering ratio is reported as median, mean, and maximum together because stalls are rare but severe: on slow 3G the median alone is 5.72% while two runs exceeded 11%, and on fast 3G the median is 0.00% while one run reached 5.99%. Reporting the median in isolation would have rendered fast 3G as "no rebuffering", which the underlying runs contradict.

Producing these figures required deciding what the received bitrate actually is, and

that question exposed a defect. The obvious source is the `BANDWIDTH` attribute of each `EXT-X-STREAM-INF` entry, but the routine that writes the master playlist was emitting that value as the sum of two target constants from its configuration, never inspecting what the encoder had produced. RFC 8216 Section 4.3.4.2 is normative on this point and states that the value "represents the peak segment bit rate of the Variant Stream", adding that where the media segments already exist the value *MUST* be derived from them. The segments do exist by the time the function runs, so the condition is satisfied and the requirement binding.

What the encoder actually produced was therefore measured directly: the size of every segment of every rendition was taken from the processed bucket and divided by that segment's own duration, read from the corresponding `#EXTINF` tag rather than assumed to be the nominal six seconds, since the final segment of each rendition is shorter and assuming otherwise would depress every figure.

Table 6.5: Declared against actual bitrate for each rendition

RENDITION	DECLARED	PEAK	MEAN	SHORTFALL
360p	464	637	514	37%
720p	1628	2148	1714	32%
1080p	4192	5390	4338	29%

Note: kbit/s; $n = 44$ segments per rendition. The shortfall column expresses how far the declared value falls below the actual peak.

The direction of the error is what makes it consequential. Because hls.js consults `BANDWIDTH` when judging whether a rendition fits the throughput it has measured, a value below the true peak invites the player to select a level the connection cannot sustain. The fast 3G profile sits precisely on that boundary: its 1678 kbit/s ceiling admits the declared 1628, and excludes the actual 1701. The prior measurement round showed the predicted consequence, with two of five runs stalling and one requiring 16.81 s to reach the first frame after eight rendition switches.

The correction computes `BANDWIDTH` from the segments themselves, taking the peak and rounding upward, since rounding down would return the declared figure to below the true value and reproduce the defect in miniature. Existing videos were repaired in place by rewriting the master playlist alone, since re-transcoding would have generated new segments and destroyed the comparison the experiment depends upon. The rendition ladder itself is unchanged.

Table 6.6: Playback quality before and after the manifest correction

PROFILE	REBUFFERING	STALLED	STARTUP MAX	BITRATE
Unthrottled	0.00% → 0.00%	0/5 → 0/5	3.29 → 5.22	1833 → 1833
Fast 3G	2.48% → 1.20%	2/5 → 1/5	16.81 → 2.39	1823 → 1551
Slow 3G	6.49% → 6.49%	3/5 → 3/5	3.83 → 3.92	1236 → 1200

Note: rebuffering is the mean across five runs, startup maximum in seconds, and average bitrate in kbit/s. Both rounds used identical parameters, and the only attribute altered between them was `BANDWIDTH`*; the manifests carried no* `CODECS` *attribute in either round, that being added afterwards.*

The correction helps in exactly one of the three conditions, and the pattern is more informative than the improvement. On fast 3G the worst-case startup delay falls from 16.81 s to 2.39 s, the whole distribution narrowing to between 2.03 and 2.39 s, while mean rebuffering halves and the number of affected runs drops from two to one. This gain is not free: the average bitrate falls by 15% and the median switch count rises from one to four, the player now adapting continuously instead of settling on a level it could not carry. Trading picture quality for continuity is a genuine trade-off and is reported here as one.

The unthrottled profile is unchanged, and serves as a control. Where bandwidth is ample the declared value constrains nothing, so a correct manifest should make no difference, and none is observed; the apparent worsening of the startup maximum from 3.29 to 5.22 s reflects a single slow run within ordinary variation, with rebuffering at zero in both rounds.

Slow 3G is unchanged for a different and more instructive reason. Its five runs reproduced the prior round to within 0.02 percentage points, mean rebuffering landing on 6.49% in both, which attests to the repeatability of the harness while confirming the correction had no effect. The explanation is that a manifest can only mislead a player that has a choice to make. At a 512 kbit/s ceiling even the lowest rung, whose actual peak is 637 kbit/s, exceeds the connection, and since hls.js cannot descend below the bottom of the ladder its behaviour is identical whatever the manifest declares. No manifest correction can address this condition; the remedy would be a lower rung or a cheaper 360p encode, which is recorded as future work rather than attempted here.

Three limitations qualify the whole of this evaluation. The average bitrate is computed from the bytes actually transferred rather than from the declared attribute, which is why it remains valid across the correction; the harness derives it by summing segment sizes and dividing by the summed `#EXTINF` durations of those same segments. The peak the

harness observes is a lower bound on the true peak, since it sees only segments the player chose to fetch, so exceeding the declared value proves a violation while remaining below it proves nothing. Finally the profiles were executed in sequence, leaving edge cache state confounded with profile order for the startup delay figures specifically, which is the likeliest explanation for fast 3G showing a lower median startup than the unthrottled profile in both rounds. The harness also exports its event log in the input format of the ITU-T P.1203 model, Mode 0; it does not compute a MOS score, and no such score is claimed here, since obtaining one requires the public reference implementation of that model.

6.3.4 Whether Content Complexity Governs the Result

Every figure reported above rests on a single source video, and one sample cannot separate a property of the system from a property of that recording. A screen capture of a mostly static browser window and a recording of a busy game scene make very different demands of an encoder, and if the preceding measurements merely describe how this system handles one particular file they generalise to nothing. Settling that question required varying the attribute that plausibly governs the outcome.

That attribute is visual complexity rather than duration. Duration need only exceed the measurement window; past that threshold a longer file consumes transcoding time without contributing information. Complexity determines how many bits the content genuinely requires, and therefore whether the rungs of the ladder separate or collapse together.

Complexity was measured rather than inferred from genre, since a static shot inside an action film is still a static shot. Each candidate was encoded at a constant quality target, sixty seconds taken from the thirty-second mark at CRF 23, and the resulting bitrate read as the measure: at constant quality the encoder holds picture fidelity and allows bitrate to follow the content, so the figure reports complexity directly. Three sources were selected on that basis and are named low, medium, and high below. They measured 151, 405, and 2298 kbit/s, a spread of 15.2× end to end with adjacent ratios of 2.7× and 5.7×.

Each source was run against all three network profiles before the next source began, so that the three profiles of every source encounter the same ordering and its effect cancels when sources are compared. Order does not cancel between the profiles of a single source, exactly as in the single-source measurement.

Table 6.7: Playback quality across three sources of differing visual complexity

METRIC	LOW	MEDIUM	HIGH
Unthrottled			
Rebuffering ratio, mean	0.00%	0.00%	0.00%
Rebuffering ratio, max	0.00%	0.00%	0.00%
Runs exhibiting a stall	0/5	0/5	0/5
Startup delay, median (s)	1.46	1.34	1.21
Average bitrate (kbit/s)	1644	1830	1664
Fast 3G			
Rebuffering ratio, mean	0.22%	2.30%	0.11%
Rebuffering ratio, max	0.60%	5.81%	0.55%
Runs exhibiting a stall	2/5	2/5	1/5
Startup delay, median (s)	1.22	1.15	1.12
Average bitrate (kbit/s)	1477	1316	1246
Slow 3G			
Rebuffering ratio, mean	4.91%	5.80%	4.84%
Rebuffering ratio, max	13.14%	13.73%	13.37%
Runs exhibiting a stall	3/5	3/5	3/5
Startup delay, median (s)	3.76	1.87	3.84
Average bitrate (kbit/s)	1070	1224	845

Note: five recorded runs per cell after two discarded warm-up runs, 120 s of playback each, at a device pixel ratio of one. Source durations are 4:23, 4:23, and 4:40, all longer than the measurement window.

Complexity does not order the result. Under fast 3G the mean rebuffering ratio runs 0.22%, 2.30%, and 0.11% across low, medium, and high, so the most demanding source is the least affected and the middle source the worst. Under slow 3G the three land within a single percentage point of one another, and under the unthrottled profile all three are identically zero. A fifteenfold difference in content complexity produces no corresponding ordering under any profile.

The explanation lies in the encoder configuration rather than in the measurement. The ladder targets fixed bitrates through average-bitrate mode, so the encoder spends its allotted budget whether the content requires it or not: the 720p rung averages 1650, 1694, and 1652 kbit/s for the three sources despite their raw complexity differing fifteenfold. Content is flattened at the point of encoding, and by the time playback occurs little remains for it to influence. The limitation recorded against the single-source measurement is therefore answered in the strongest form available, since adding sources leaves the result unchanged and that is now established by measurement rather than assumed.

A second finding emerged from the rendition ladder, and it governs how the unthrottled row of every table in this section must be read. The player configures hls.js with `capLevelToPlayerSize`, which restricts the selectable renditions to those needed to

fill the video element, and hls.js scales that element's dimensions by the device pixel ratio before comparing. The watch page holds the player column at roughly 880 px on any display, because the layout caps the page at 1320 px and reserves 360 px for the sidebar. At a device pixel ratio of one the player therefore requires at most 1280×720, and the 1080p rung is unreachable: across all nine measurements above it received one segment in total. Contemporary laptops commonly report a ratio between 1.25 and 2, at which the same element is treated as 1320 to 1760 px wide and the 1080p rung is selected. The unthrottled row was consequently measured a second time at a ratio of two.

Table 6.8: Unthrottled playback at device pixel ratio one against two

SOURCE	DPR 1	DPR 2	RUNGS, DPR 1	RUNGS, DPR 2
Low	1644	4318	360p, 720p	360p, 1080p
Medium	1830	4312	360p, 720p	360p, 1080p
High	1664	4206	360p, 720p	360p, 720p, 1080p

Note: average bitrate in kbit/s, measured over five recorded runs per cell. Rebuffering was 0.00% in every run of both configurations. The medium source additionally fetched a single 1080p segment at a ratio of one, which is treated here as unused.

Average bitrate rises by a factor of 2.5 to 2.6 while rebuffering remains at zero and the median startup delay moves by at most 0.34 s. The unthrottled profile as measured at a ratio of one was therefore bounded by the resolution ceiling rather than by the network, and the delivery path sustains two and a half times the throughput without a single stall. Neither configuration exercises the whole ladder: at a ratio of one the 1080p rung is unreachable, while at a ratio of two the low and medium sources ascend directly from 360p to 1080p and bypass 720p entirely. The middle rung is thus seldom selected on an unconstrained connection, and serves principally on the fast 3G profile.

Raising the ceiling also closed a caveat carried by the earlier measurement. The peak bitrate the harness observes is a lower bound on the true peak, since it sees only the segments the player chose to fetch, so remaining below the declared value proves nothing. At a ratio of two the 1080p rung was fetched in quantity, and the observed peak matched the declared `BANDWIDTH` exactly for all three sources, at 4 601 399, 5 389 584, and 5 495 113 bit/s respectively. Two computations performed independently, one by the packager from segment sizes in the processed bucket and the other by the harness from bytes received over the network, agree to the bit. The declared value is therefore attained by a real segment rather than overstated, although the harness still cannot exclude a heavier segment among those it never fetched.

Two limitations attach to this comparison specifically. The 120 s window covers

approximately the first twenty of the forty-four segments of each source, so statements about peak bitrate concern the opening of each file rather than the whole of it. And variance under fast 3G was markedly higher than under either neighbouring profile: two of the three sources produced a bimodal startup distribution, clustering near 1.2 s and again near 7.5 s, while the third remained uniform at 1.1 s. Fast 3G is the only profile whose ceiling falls between two rungs rather than above all of them or below all of them, which would place the adaptation algorithm at a decision boundary, but the third source contradicts that account and the cause was not established here.

6.4 Transcoding Pipeline Performance

Pipeline latency is measured from the completion of the upload to the moment the record reaches the `READY` state, which corresponds to the interval a user actually waits. This interval encompasses the S3 event delay, the time spent queued in SQS, container startup on Fargate, FFmpeg execution, and the upload of results.

The compute allocation of each run was verified after the fact rather than taken from the label the measurement script recorded, since a mislabelled run would silently invalidate the comparison that follows. Querying the job definition revision each job actually executed against resolved every run in Table 6.9 to the 1 vCPU / 2 GB revision, and every run in the comparison after it to the 4 vCPU / 8 GB revision.

Table 6.9: Transcoding pipeline latency by input size

SIZE	DURATION	UPLOAD (S)	PIPELINE (S)	REALTIME FACTOR
100 MB (n=3)	120 s	3.13	475.80	3.97
500 MB (n=1)	600 s	13.13	2281.25	3.80
1 GB (n=1)	1600 s	72.00	5917.03	3.70

Note: measured at the 1 vCPU / 2 GB job configuration. Pipeline and upload columns are the arithmetic mean across replications. Realtime factor is pipeline time divided by video duration, so a value above 1.0 means transcoding took longer than the video's own runtime.

Two observations follow from the table. The realtime factor stays within a narrow band of 3.70–3.97× across a more than thirteen-fold range of content duration, which indicates that the pipeline scales essentially linearly with duration at fixed compute. And for the 100 MB size, the three individual runs of 346.32, 539.80 and 541.28 s did not show the monotonic cold-start slowdown the measurement protocol anticipated, the first

run being in fact the fastest of the three; at this scale Fargate Spot capacity variability appears to dominate over container image pull cost, rather than any first-run-specific effect.

The precision of two rows is limited. The 500 MB and 1 GB sizes were each measured once, because a full run at those sizes occupies between 30 and 100 minutes of AWS Batch time and repeated sampling was costly within the time and budget available. Chapter 7 returns to this as a threat to those two rows specifically.

A separate experiment raised the container allocation from 1 vCPU / 2 GB to 4 vCPU / 8 GB and repeated the measurement on the same two inputs. Pipeline time fell from a mean of 475.80 s to a mean of 115.56 s for the 100 MB input (3 runs each), and from 5917.03 s (1 run) to a mean of 1078.70 s for the 1 GB input (2 runs: 1080.29 s and 1077.10 s). Taken at face value these are speedups of 4.12× and 5.49× respectively.

Both figures exceed the 4× that the change in vCPU count alone could account for, and that observation deserves scepticism rather than celebration. Amdahl's Law bounds the achievable speedup *below* the resource ratio, because the serial portion of an encoder, entropy coding and rate control among others, does not parallelise; the multi-threaded H.264 literature reports exactly this sub-linear pattern, with Sankaraiah *et al.* measuring 1.97×/3.96×/7.71× at 2/4/8 threads [10] and Chen *et al.* 4.7–8.6× across six Cell SPEs [11]. A measured result above the resource ratio therefore indicates that something other than thread-level parallelism is contributing, and two confounds are visible in the data.

The first is that the experiment did not vary vCPU count in isolation: memory was raised from 2 GB to 8 GB at the same time, so any benefit from reduced memory pressure during the concurrent encoding of three renditions is folded into the same measurement. The second, and more serious, is the instability of the 1 vCPU baseline. The three 100 MB runs at 1 vCPU span 346.32 s to 541.28 s, a spread of 56.3%, whereas the corresponding 4 vCPU runs span only 24.0%. The speedup estimate is consequently very sensitive to which baseline statistic is chosen: 3.29× comparing the fastest run of each configuration, 4.12× comparing means, 4.15× comparing slowest runs, and 4.86× comparing medians. Only the first of these is consistent with the sub-linear bound the theory predicts. The 1 GB comparison rests on a single 1 vCPU observation and so admits no variance estimate at all.

The conclusion this data supports is therefore narrower than the headline numbers suggest: raising the allocation from 1 to 4 vCPU reduced pipeline latency by roughly a factor of four on this workload, but the present sample is too small and too noisy to establish whether the true scaling is sub-linear as theory requires, and the simultaneous

memory change means the experiment does not isolate the effect of vCPU count. Establishing the scaling behaviour properly would require repeated 1 vCPU baselines, since the variance there rather than in the 4 vCPU measurements is what limits the conclusion, together with a separate arm that varies memory alone. This is recorded as future work in Section 7.4 rather than presented as a settled result.

Cost per video, by contrast, is robust to this uncertainty: AWS Batch on Fargate bills per vCPU-second, so roughly four times the hourly rate for roughly one quarter of the duration leaves the total charge per video essentially unchanged. The practical implication is that the 4 vCPU configuration buys a substantially lower waiting time for the user at close to no additional cost, which holds regardless of whether the exact speedup is 3.3× or 4.9×.

Note: each size should be measured at least three times, and the first run reported separately, because it includes the cost of pulling the container image and is therefore substantially slower than the steady state.

6.5 Chunked Transcoding of Long Videos

The single-container design evaluated in Section 6.4 transcodes a video at a roughly constant fraction of its own runtime, and one AWS Batch job is bounded by the two-hour attempt timeout set in its job definition. A requirement that the platform accept videos of four hours or more cannot be met by that path: a five-minute video took 26.7 minutes as a single job, which extrapolates to about twenty-one hours for four hours of content. This section evaluates the second execution path that was added for that reason.

The path applies to videos of five minutes or longer, a threshold chosen from the measurements in Table 6.10. The job started by the upload event then becomes a planner. It reads the container structure of the source over HTTP range requests, divides the video into chunks whose length is a whole number of six-second groups of pictures, and submits child jobs to AWS Batch: two audio jobs, each encoding one audio bitrate once for the whole video; an array job whose elements each encode one chunk at every rendition; and a finalizer that starts when every chunk has finished, concatenates the playlists, measures the bandwidth of each rendition from the produced segments, extracts the thumbnail, runs content moderation and sets the record to `READY`. The array job is placed in a second, lower-priority queue that shares the compute environment, so a short video submitted behind a long one is served before the remaining chunks rather than after them. The planner chooses the chunk length so that each concurrent slot receives about three chunks, between one and five minutes long, and falls back to the single-job path when the source

cannot be split safely, for example when its frame rate is variable or its audio and video start more than 0.25 seconds apart. Chunk boundaries were verified by analysing the packets of the produced renditions for gaps and overlaps, and the check passed in every run reported below.

All measurements ran in the staging environment, which uses the same container image, job definitions and compute settings as production but a separate database and separate storage buckets. Capacity is the Fargate vCPU quota available to the compute environment: 8 vCPU of Fargate Spot, which is the production quota, or 16 vCPU when an On-Demand environment of 8 vCPU was attached behind it. Each chunk runs in a 4 vCPU / 8 GB task, the size selected by the comparison in Table 6.11. Latency is measured, as in the previous section, from the moment the upload finishes to the moment the record reaches `READY`, taking the return of the upload command as the end of the upload. The 25-minute source is a 1080p H.264 file of 2.75 GiB with an AAC track. The four-hour source was built by repeating a 1,600-second 1080p test-pattern clip nine times and adding a 128 kbit/s tone, and was uploaded with the S3 multipart protocol, which the browser path also uses for files larger than 512 MiB.

Table 6.10: Single-container path against chunked path for videos of five to fifteen minutes

VIDEO LENGTH	SINGLE JOB (MIN)	CHUNKED (MIN)	SPEED-UP
5 min	26.7	11.9	2.2
10 min	≈ 53.0	27.2	≈ 1.9
15 min	≈ 80.0	17.4	≈ 4.6

Note: values marked ≈ for the single-container path are extrapolated at 5.3 times the video duration from the measured five-minute run. The chunked values come from three videos competing for one 8 vCPU quota, so they overstate what a lone submission would see.

Table 6.10 shows why the threshold sits at five minutes. For a five-minute video the chunked path finished in 11.9 minutes against 26.7 minutes as a single job, about 2.2 times faster, even though planning, audio encoding and finalisation take roughly 4.4 minutes of fixed cost. Below that length the fixed cost would be more than half of the processing time, so shorter videos keep the single job.

Table 6.11: A 25-minute video on the chunked path, by chunk configuration and capacity

N × VCPU	FARGATE	VCPU	SLOTS	PHASE (MIN)	READY (MIN)	VCPU-S
5 × 1	Spot	8	8	n/a	30.1	n/a
5 × 1†	Spot	8	8	32.6	36.3	7 117
23 × 1	Spot	8	8	19.2	23.6	7 270
12 × 2	Spot	8	4	16.3	20.1	6 816
6 × 4	Spot	8	2	16.8	20.4	6 792
6 × 4	Spot	8	2	14.4	18.0	6 363
6 × 4	On-Demand	8	2	21.7	26.0	10 182
6 × 4	On-Demand	8	2	18.2	22.5	7 852
12 × 4	Spot + OD	16	4	11.5	15.3	8 827
12 × 4	Spot + OD	16	4	10.2	14.7	8 159

Note: the first four rows were timed from the creation of the first job and the later rows from the end of the upload, which differ by about half a minute. The vCPU-seconds of the first five rows count chunk jobs only; the later rows count every job of the video. Rows are in chronological order of measurement. The first column gives the number of chunks and the vCPU of each; † marks the run on the production environment. PHASE is the time from the first chunk start to the last chunk end, SLOTS the number of chunks encoded at once, VCPU-S the vCPU-seconds, and OD stands for On-Demand.

Table 6.11 follows the same 25-minute video through the changes made to the chunked path. The first deployment cut the video into five chunks of five minutes on 1 vCPU tasks and took 36.3 minutes; five chunks used only five of the eight available vCPU, and chunks of identical size ran between 18.2 and 32.6 minutes because Fargate Spot hosts are not uniform. Choosing the chunk length from the capacity, which produced 23 chunks, reduced the time to 23.6 minutes, and moving the chunks to 4 vCPU tasks reduced it to 20.4 minutes and, on a later repeat, 18.0 minutes; a 4 vCPU task processed a second of video 4.7 times faster than a 1 vCPU task. Twelve 2 vCPU tasks gave the same time and the same vCPU-seconds as six 4 vCPU tasks, which is why the larger size was kept. Doubling the capacity to 16 vCPU shortened the video to 14.7 and 15.3 minutes, a factor of only 1.2, for 1.3 to 1.4 times the vCPU-seconds: planning, audio encoding and finalisation take about three minutes whatever the capacity, and each of the twelve smaller chunks pays a container start-up. For a single video of this length, additional capacity is therefore close to saturation.

Table 6.12: A four-hour video at two capacities

STEP (MINUTES)	16 VCPU	8 VCPU
Planner (Lambda submit to child jobs)	0.4	0.4
Audio 64 kbit/s and 128 kbit/s (in parallel)	9.6	4.7
48 chunk jobs, 4 vCPU each	71.8	125.9
Finalizer (playlists, thumbnail, moderation, READY)	2.3	3.2
Upload complete to READY	**85.3**	**135.3**
Compute (vCPU-hours)	17.3	15.1

Table 6.12 reports the requirement that motivated the path. At the production capacity of 8 vCPU a four-hour video reached `READY` 135.3 minutes after its upload finished, and with 16 vCPU, which doubles the number of chunks encoded at once, 85.3 minutes. Both are far inside the two-hour limit of a job, because no chunk ran longer than seven minutes. The first measurement of the path, taken with 1 vCPU chunks at 8 vCPU, took 185.0 minutes, so the later changes shortened the result at production capacity by 27%. Capacity helped the long video more than the 25-minute one, 1.6 times against 1.2, because 48 chunks keep every slot busy through many rounds and the fixed phases are a small share of the total. The compute used was 15.1 vCPU-hours at 8 vCPU and 17.3 at 16 vCPU, so doubling the capacity cost about 15% more compute for 37% less time.

Table 6.13: Four 25-minute videos submitted together, by capacity

VCPU	ORDER	START (MIN)	END (MIN)	READY (MIN)
8	1	2.8	19.7	21.0
8	2	18.4	37.1	38.7
8	3	35.5	58.5	62.9
8	4	52.2	71.9	73.3
16	1	3.0	16.0	17.7
16	2	2.7	24.6	25.7
16	3	22.2	35.0	36.1
16	4	32.0	43.3	44.3

Note: in the 8 vCPU rows the four videos were uploaded together over one connection and timed from the first job creation. In the 16 vCPU rows the arrival was produced by server-side copies of one uploaded file, so all four events reached the pipeline within a second. START and END are the first chunk start and the last chunk end, and ORDER is the order of completion.

When four 25-minute videos arrived together (Table 6.13), 8 vCPU finished them one after another at 21.0, 38.7, 62.9 and 73.3 minutes, a throughput of 1.4 times real time,

and 16 vCPU finished them at 17.7, 25.7, 36.1 and 44.3 minutes, 2.3 times real time. Capacity therefore shortened the batch by a factor of 1.65 for twice the vCPU, closer to proportional than for a single video, because the additional slots were always filled with chunks of the other videos. The order of completion follows the order of arrival: the queue is first-in first-out within a priority, so the first video is delayed least and the last most.

Table 6.14: Simultaneous submissions at 16 vCPU

BATCH	VCPU	READY	ALL (MIN)	P50 (MIN)	P95 (MIN)	PER MIN	VCPU-H
4 × 25min-1080p	16	4/4	44.3	36.1	44.3	0.09	9.2
100 × 60s-1080p	16	100/100	35.8	17.9	33.7	2.80	8.3

Note: ALL is the time of the whole batch, P50 and P95 the median and 95th percentile of the latency of a video from its arrival, PER MIN the videos completed per minute, and VCPU-H the vCPU-hours used.

To reach the load level named in the evaluation plan, one hundred 60-second clips were submitted at once at 16 vCPU (Table 6.14). All one hundred reached `READY` without a failure. The whole batch took 35.8 minutes, the median clip waited 17.9 minutes and the 95th percentile 33.7, and the batch drained at 2.8 videos per minute for 8.3 vCPU-hours of compute. The fastest clip needed 3.7 minutes, which is the time of a single job running alone, so the rest of the latency is queueing: sixteen one-vCPU jobs run at a time and one hundred clips need about six rounds. The queue absorbed the burst without losing any work, and the capacity quota rather than the architecture set the drain rate.

Fargate On-Demand did not make the work faster. The same 25-minute video on 8 vCPU of On-Demand took 26.0 and 22.5 minutes against 18.0 on Spot, with chunk medians of 414 and 313 seconds against 255 seconds, so On-Demand showed no advantage and, at a higher price per vCPU-second, was several times more expensive per video. Two On-Demand runs and one Spot run do not establish that On-Demand is slower, because the host hardware behind a Fargate task is not guaranteed, but they give no reason to prefer it other than as overflow when the Spot quota is exhausted. Spot capacity was also reliable: in the seven days of AWS Batch job history queried at the end of the measurements, 2 of 370 job executions (0.5%) had been interrupted, and both were retried successfully by AWS Batch.

Several limits apply to these figures. Most rows are single runs, and chunks of identical size on Fargate Spot varied by up to 1.8 times within one job in the first deployment, so differences of about ten percent between two rows should not be read as an effect. The four-hour source is a synthetic test pattern at 5.35 Mbit/s, which may encode faster than camera footage, although its chunks ran at 0.9 seconds per second of video against 1.0 for the 25-minute file, which suggests the bias is small. The 16 vCPU capacity exceeds the quota granted for production at the time of the measurements, so those rows describe what a quota increase would give rather than the deployed system, and the staging environment served no other traffic during the runs, so production contention is not represented.

6.6 Load Testing

Scalability is assessed by submitting concurrent uploads with a virtual user count ramping to one hundred, verifying that the queue retains all work and that AWS Batch scales to process it. Upload payloads in this test are synthetic (a small fixed byte string rather than a real video file), because the object here is to exercise the upload API, S3 event notification, SQS queue, and Batch job submission path under concurrent load, not to measure transcoding quality. Job outcome is therefore not the metric of interest; queue behaviour is.

One caveat must be stated before the results, because it bounds what they can be taken to show. Inspecting the failure reasons of the submitted jobs afterwards revealed that they terminated at container initialisation with `ResourceInitializationError`, not at the FFmpeg stage as the synthetic payload would have caused. The cause was a defect in the Terraform configuration: the Secrets Manager entry holding the notification e-mail password was created as an empty container without a value whenever e-mail was left unconfigured, while the Batch job definition still declared that secret, so every task failed before its entrypoint ran. The defect is described and its correction recorded in Section 5.5. Its consequence for this experiment is specific: the control-plane observations below, that Amazon SQS retained and forwarded every message and that AWS Batch admitted exactly its configured number of concurrent tasks, are unaffected, because those mechanisms operate before and independently of the container's entrypoint. The *rate* at which the backlog cleared, however, reflects how quickly Fargate provisions and fails a task, not how quickly it completes real transcoding work, and must not be read as a throughput figure for production load. This gap is closed, though not with the same measurement precision, in Section 6.6.2.

Table 6.15: Concurrent upload stress test results (API layer)

METRIC	VALUE
Total requests	100
Throughput (requests/s)	15.52
Failure rate	0%
Mean response time (ms)	2388.52
95th percentile (ms)	3317.79
Peak virtual users	50

Note: produced by a second load generator, written against native `fetch` *in Node.js. All 100 uploads (initiate, direct-to-S3 PUT, confirm) succeeded at the API layer.*

6.6.1 Cross-Validation with an Independent Tool

A single load generator can produce a misleading result if the generator itself, rather than the system under test, is the bottleneck. The experiment was therefore repeated with k6, an industry-standard load testing tool implemented in Go, driving the same three-step upload sequence against the same deployed endpoint at the same concurrency (50 virtual users, 100 total iterations). The k6 scenario uses the `shared-iterations` executor rather than the time-based ramp that the script originally specified, so that the total unit of work is fixed at exactly 100 uploads and is therefore directly comparable rather than merely similar.

Table 6.16: Cross-validation of the load test across two independent tools

METRIC	NODE.JS	K6
Upload cycles completed	100	100
Failure rate	0%	0%
Batch jobs queued (RUNNABLE)	92	92
Batch jobs admitted (STARTING)	8	8
SQS messages remaining	0	0
Queue drain, 76 jobs (s)	416	417
Consumption rate (jobs/min)	10.96	10.94

The two tools agree exactly on every infrastructure-level observation. Immediately after each burst the queue held 92 jobs waiting with 8 admitted for execution, and Amazon SQS reported zero messages outstanding. The drain rate agrees to within 0.24%: measuring from the common reference point at which 76 jobs remained queued until the queue emptied took 416 s in the Node.js run and 417 s in the k6 run, a consumption rate of 10.96 against 10.94 jobs per minute. Reproducing both the 92/8 split and the drain

rate across two independently implemented generators running on different runtimes is considerably stronger evidence than either run alone that the eight-job ceiling and the resulting throughput are properties of the compute environment's configuration namely `max_vcpus` = 8 divided by `job_vcpu` = 1, rather than artefacts of how one particular client paced its requests.

The latency figures, by contrast, are *not* directly comparable, and reporting them side by side without qualification would be an error. The Node.js script times each complete upload cycle, so its mean of 2388.52 ms covers all three HTTP requests together; k6's `http_req_duration` times each HTTP request individually, giving a mean of 670.03 ms per request across 300 requests. Scaling the latter by the three requests per cycle yields 2010.09 ms, within 15.8% of the Node.js figure, close enough to corroborate it, and the residual gap is consistent with k6 running inside a Docker container with an additional network hop that the natively executed Node.js script did not incur. Throughput agrees on the same basis once k6's virtual-user initialisation is accounted for: the raw 100 cycles over 9.83 s understates the steady-state rate, because no iteration completed during the first four seconds, whereas the 100 cycles that completed in the remaining 5.8 s correspond to 17.24 cycles per second against the Node.js script's 15.52.

The queue and compute layer were observed separately by polling the AWS Batch job queue every 25 seconds after the 100 uploads completed. Two data points anchor this observation: immediately after the burst, 92 jobs were queued in the `RUNNABLE` state with 8 already `STARTING`, and the Amazon SQS queue itself reported zero messages remaining, meaning the Lambda job submitter had already drained every event into AWS Batch within seconds of the S3 notifications firing. The `STARTING` count held at 7–8 throughout the drain, matching `max_vcpus` = 8 divided by `job_vcpu` = 1 exactly, which confirms the compute environment enforced its configured concurrency ceiling rather than either exceeding it or under-utilising it. The backlog of 92 queued jobs reached zero 456 seconds after the burst, and no message was lost or left stuck. Subject to the caveat stated above, this is the direct evidence for the claim that opened this section: Amazon SQS retained every unit of work submitted faster than the compute layer could consume it, and AWS Batch admitted work strictly at, and never above, its configured limit while consuming that backlog.

6.6.2 Validating Real Transcoding Throughput

The caveat stated at the start of this section was closed once the Secrets Manager defect was corrected (Section 5.5) and the concurrency test was re-run with a genuine three-

second H.264 source file in place of the synthetic byte string that every earlier run in this section used. The scenario, executor, and concurrency were otherwise unchanged from Table 6.16: fifty virtual users driving the `shared-iterations` executor to exactly one hundred upload cycles against the same production endpoint.

All three hundred HTTP requests succeeded, covering initiate, direct-to-S3 PUT, and confirm across the one hundred cycles, for a throughput of 23.47 requests per second and a 95th-percentile request duration of 1827 ms, both consistent with the request-level figures in Table 6.16. The decisive difference is downstream of the HTTP layer: every one of the resulting one hundred Batch jobs reached `SUCCEEDED`, and none failed. Two point-in-time samples of the job queue, taken roughly ten minutes apart while the backlog drained, showed 18 succeeded / 3 running / 84 queued shortly after the burst and 28 succeeded / 5 running / 74 queued about ten minutes later, consistent with the same seven-to-eight-way concurrency ceiling established in Section 6.6.1 and confirming that ceiling now gates genuine FFmpeg work rather than a task that fails at initialisation. What it establishes, which no earlier run in this chapter could, is that the queueing and admission-control behaviour validated in Section 6.6.1 sits in front of a pipeline that actually completes real transcoding work at that concurrency rather than one that only queues and admits it.

6.6.3 Drain Rate Under Real Transcoding Load

That run established that the work completes but not how fast the backlog clears, because it was not instrumented with the twenty-five-second polling cadence that produced the synthetic drain curve. Repeating it with that cadence in place closes the last measurement gap and, more usefully, makes the two curves directly comparable: same scenario, same executor, same fifty-way concurrency and one hundred uploads, differing only in whether the payload is a real video.

The HTTP layer behaved as before, with three hundred requests, no failures and 33.12 requests per second, and all one hundred Batch jobs again succeeded, with none failing. Independently of the Batch job states, all one hundred video records were confirmed to have reached `READY` in the database, which verifies that the jobs did the work rather than merely exiting zero. Table 6.17 reports the comparison.

Table 6.17: Backlog drain under synthetic and real payloads

	SYNTHETIC	REAL VIDEO
Jobs submitted	100	100
Jobs succeeded	0	100
Drain duration (s)	456	826
Drain rate (jobs/min)	13.16	7.26
Peak concurrent `RUNNING`	0	8
Implied wall-clock per job (s)	n/a	66.1

Note: both columns sampled every 25 s. The synthetic column records no successes and no `RUNNING` observations because those jobs failed at FFmpeg almost immediately, which is what that experiment intended. Implied wall-clock per job is the drain duration multiplied by the observed concurrency and divided by the job count.

The real backlog drains at 7.26 jobs per minute against 13.16 for the synthetic one, so doing the work costs a factor of 1.8 in throughput relative to merely admitting and rejecting it. The concurrency ceiling is confirmed rather than inferred: `RUNNING` peaked at exactly eight, matching `max_vcpus` divided by the per-job vCPU allocation, and stayed there through the linear portion of the curve. The synthetic run could not establish this, because with jobs failing on startup the `RUNNING` count was zero at every sample and only `STARTING` was observable.

The implied per-job figure is where the number becomes interesting rather than merely precise. At eight-way concurrency, 826 seconds for one hundred jobs means roughly 66 seconds of wall-clock per job, and that is for a three-second video. Section 6.4 measured a 100 MB input at around 476 seconds, so the fixed overhead visible here is not a small fraction of that, but for short content it is almost the entire cost. What the pipeline spends on a three-second clip is overwhelmingly container provisioning, image pull and result upload rather than encoding, which explains why the realtime factor reported earlier stays in a narrow band across input sizes: the variable cost scales with duration while a fixed cost of roughly a minute is paid regardless. For a platform whose median upload is short, that fixed cost, not FFmpeg's throughput, is what determines how long a user waits.

6.7 Cost Analysis

The cost comparison contrasts a traditional deployment, in which an Amazon EC2 instance runs FFmpeg continuously, against the serverless container architecture. The workload assumption is three hundred videos per month averaging five minutes each,

with a mean transcoding time of forty-five seconds, giving 3.75 hours of actual processing per month.

Table 6.18: Monthly operating cost comparison (ap-southeast-1)

COMPONENT	EC2 (ALWAYS ON)	SERVERLESS	SAVING
Compute	`c5.xlarge` for 730 h: $124.10	Fargate Spot for 3.75 h: $0.05	99.96%
Storage	EBS 100 GB plus S3 50 GB: $9.25	S3 with Glacier lifecycle: $1.45	84.32%
Queueing	Self-hosted broker	SQS and Lambda: $0.01	n/a
Total	**$133.35**	**$1.51**	**98.87%**

Content delivery cost is excluded from Table 6.18 because it is neutral with respect to the architectural choice: the volume of data served to viewers is identical whichever compute model produces the renditions. Attributing a CloudFront charge to the EC2 column while crediting the serverless column with the free tier would overstate the saving. When delivery is included on both sides at post-free-tier rates, the total becomes $145.35 against $13.51, a saving of approximately 90.7%, which is the more conservative figure and the one that should be quoted for total cost of ownership at scale.

The saving originates entirely in the elimination of idle capacity. Under the stated assumptions the always-on instance is billed for 730 hours while performing useful work for only 3.75 of them, a utilisation of roughly 0.5%. It follows that the advantage is not universal: as sustained utilisation rises, per-second Fargate billing approaches and eventually exceeds the cost of a reserved instance. The architecture is therefore best suited to workloads that are genuinely bursty, which is the case for user-generated video uploads.

Chapter 7

Conclusion and Future Work

7.1 Summary of Achievements

This project set out to build a video sharing platform whose transcoding subsystem runs on a serverless container and event-driven architecture, and to evaluate whether that choice delivers the cost and scalability benefits it promises.

The functional platform was completed. Users can register, authenticate, upload videos directly to Amazon S3 through pre-signed URLs, watch them with adaptive bitrate playback across three renditions, control whether each video is public, unlisted or private, manage their channel, search and filter the catalogue, and interact through likes and comments.

The transcoding pipeline operates without manual intervention. An upload emits an S3 event that is buffered in Amazon SQS and dispatched by an AWS Lambda function to AWS Batch, where a container packaging FFmpeg produces the renditions, the master playlist, and a thumbnail before terminating. Conditional database writes prevent a duplicate delivery from overwriting a completed job, failed jobs are retried by AWS Batch and, when they exhaust their attempts, raise an alert carrying the reason for the failure rather than terminating silently. Both mechanisms replaced earlier designs that held only for an execution mode the deployed system does not use; the reasoning is set out in Section 4.2 and Section 5.5.

The platform also processes long videos. Videos of five minutes or longer are divided by a planner job into chunks that an array of AWS Batch jobs encodes in parallel and a finalizer job merges, while shorter videos keep the single-job path. Measured on the deployed pipeline, a four-hour video reached the `READY` state 135 minutes after its upload finished at the production capacity of 8 vCPU and 85 minutes at 16 vCPU, and one hundred 60-second clips submitted at once were all processed without a failure

(Section 6.5).

Content protection was implemented in two layers. Origin Access Control keeps the storage bucket entirely private, and CloudFront Signed Cookies ensure that private videos are rejected at the edge even when the exact manifest URL is known. This closes a gap that would otherwise have made the private visibility setting ineffective in practice, since the API can restrict its own responses but cannot restrict a content delivery network.

The infrastructure is fully described as code in eleven Terraform modules across two environments, and seven GitHub Actions workflows automate linting, unit testing, static security analysis, container scanning, infrastructure validation, and deployment. The security gate blocks merges on fixable critical findings rather than merely reporting them.

The automated test suite grew to 209 tests across fourteen suites. Its practical value was demonstrated when it caught a defect in the cookie signing implementation that would have prevented every private video from playing once deployed. A second defect in that same subsystem, which only appeared once the distribution was live, is described in Section 5.5.

7.2 Strengths of the System

The strongest property of the design is the separation between the application tier and the media path. The backend API issues pre-signed URLs, records metadata and answers queries, and never handles video bytes, which flow directly between the browser, Amazon S3 and CloudFront. The API's resource consumption is therefore independent of video size and upload concurrency, and a failure in the processing tier leaves already-published videos playable.

The processing tier costs nothing while idle. Because transcoding runs on AWS Batch over Fargate rather than on a reserved instance, compute capacity exists only while a job runs. Section 6.7 estimates the monthly operating cost of this arrangement at $1.51 against $133.35 for an always-on instance sized for the same workload, a difference that originates entirely in the elimination of idle capacity.

The same property extends to long videos. Chunk jobs are ordinary AWS Batch jobs on the same compute environment, so additional vCPU quota translates directly into lower latency: a four-hour video took 135 minutes at 8 vCPU and 85 minutes at 16 vCPU, and no chunk came near the two-hour limit of a job (Section 6.5).

Content protection is enforced at the edge rather than only in the API. Origin Access Control keeps the storage bucket private and CloudFront Signed Cookies reject unau-

thorised requests before they reach it, which is what makes the private visibility setting meaningful, since an API can restrict its own responses but cannot restrict a content delivery network. The mechanism was verified against the deployed system rather than in code alone, as Section 5.5 records.

The infrastructure is reproducible. Eleven Terraform modules describe two environments in full, and seven GitHub Actions workflows automate linting, unit testing, static security analysis, container and dependency scanning, infrastructure validation and deployment. The security gate blocks a merge on fixable critical findings rather than merely reporting them, and the scanner itself is pinned to a release commit rather than to a moving branch.

Finally, the performance claims in this report are measured rather than asserted. Time-to-First-Frame was measured from six AWS regions and from a consumer connection in Vietnam, and led to two corrections of the CloudFront price class that reduced the median latency seen from Vietnam by a factor of 6.6 and the median seen from Brazil by a factor of 32. Transcoding throughput was measured under one hundred concurrent uploads of real video and cross-validated with two independently implemented load generators. And the automated suite of 209 tests across fourteen suites caught a signing defect that would otherwise have prevented every private video from playing.

7.3 Limitations

Several limitations remain in the delivered system.

The bitrate ladder is fixed at three rungs regardless of content. As the literature reviewed in Section 2.2 shows, the optimal ladder is content-dependent, so a fixed ladder wastes bandwidth on simple content while under-serving complex content. Netflix reported bitrate reductions between 28% and 38% at equivalent perceptual quality by optimising the ladder per shot [7], which indicates the magnitude of the opportunity forgone.

Parallelism is bounded by the account quota and by fixed per-job costs. Chunked transcoding converts capacity into lower latency for long videos, but the gain flattens for shorter ones: a 25-minute video finished in 18.0 minutes at 8 vCPU and in about 15 minutes at 16 vCPU, a speed-up of only 1.2, because planning, audio encoding and finalisation take about three minutes regardless of capacity and each chunk pays a container start-up (Section 6.5). Production runs on a Fargate Spot quota of 8 vCPU, so a burst of long videos is served one after another: four 25-minute videos submitted together finished after 73 minutes, and a request for a larger quota had not been granted when the

measurements ended. Videos shorter than five minutes keep the single-container path, where processing time still scales linearly with duration; Section 6.6.3 measured roughly 66 seconds of wall-clock time per job for a three-second clip, which shows how far fixed overhead dominates for short content.

Signed Cookies remain valid for two hours after issuance, so switching a video from public to private leaves viewers holding unexpired cookies with access until those cookies lapse. Immediate revocation would require rotating the signing key or introducing an edge function that performs a per-request check. The system also implements no digital rights management: an authorised viewer can capture the decrypted stream or share their cookies, and preventing this is outside the scope of the techniques used here.

Rate limiting and view deduplication hold their state in process memory. This is correct for the current single-instance deployment but would need to move to a shared store such as Redis before the backend could be scaled horizontally, since each instance would otherwise enforce its own independent limits. The backend itself remains one process on one instance in a single region and is the only single point of failure in the system, the transcoding and delivery tiers being managed services that scale and recover on their own.

The channel management interface exposes metadata modification through an in-place editing modal, allowing uploaders to update the title, description, category, and visibility of an existing video without losing accumulated views, comments, or transcoding artifacts. However, tags management remains confined to the initial upload form, and thumbnail replacement is not yet supported in the interface; changing a video's poster requires administrative intervention or re-uploading.

The long-video measurements are narrower than their headline figures in one respect as well: the four-hour results are one run per capacity on a synthetic source, a test pattern repeated nine times, and the 16 vCPU rows describe a quota that production does not currently have (Section 6.5).

Two experimental results are narrower than the headline figures suggest. The comparison between 1 and 4 vCPU allocations rests on a 1 vCPU baseline whose three runs span 56%, so the estimated speedup varies between 3.3 and 4.9 times depending on which statistic is compared, and memory was raised at the same time as the vCPU count, so the experiment does not isolate either variable. The delivery latency figures were collected from probes running inside AWS regions rather than on residential connections, so they describe edge proximity under favourable network conditions rather than what a home viewer would experience.

The two-language interface has one gap of the same kind. A viewer's language

choice is held in the browser and sent to the API on each request, which is sufficient for everything the interface and the API produce, but the notification e-mails announcing that a video is ready or has failed are sent from an AWS Batch job long after the request that started it. That job has no request to read a language from, and the account record stores no language preference, so those two e-mails are sent in both languages. Storing the preference on the user document would let them be sent in one, and is the correct fix rather than the one adopted here.

Finally, the CloudFront distributions are provisioned in a second AWS account, because the primary account was not permitted to create them. The arrangement is invisible to viewers and changes nothing about the security property being provided, but it splits the infrastructure across two sets of credentials and carries operational overhead that a verified account would not.

7.4 Future Work

Two measurements should still be repeated before the experimental results are considered settled. The Time-to-First-Frame measurement has been repeated against the live distribution (Section 6.3.1) and then from several client locations (Section 6.3.2), which closed that item and, in doing so, exposed a further limitation that has since been corrected: viewers in South America and Oceania fell outside the configured price class and were served across an ocean until both distributions were moved to `PriceClass_All`. What remains open is that every probe ran inside an AWS region, so the figures describe edge proximity under favourable network conditions rather than residential connections; comparing the two Vietnamese samples suggests the last mile costs roughly a factor of two, but one pair of observations is not a basis for that claim. The concurrency test re-run with genuine video payloads (Section 6.6.2) confirmed that every job succeeds under load, but it was not instrumented with the same twenty-five-second polling cadence as the original synthetic run, so a precise drain-rate figure for real transcoding throughput, as opposed to confirmation that throughput is non-zero and failure-free, remains to be measured; repeating the same short-clip payload with that polling protocol in place would close this. The vCPU scaling comparison should also be repeated with more replications of the 1 vCPU baseline and with memory held constant, since Section 6.4 showed the present estimate to be dominated by baseline variance and confounded by a simultaneous change in memory allocation.

The most valuable functional extension would be per-title bitrate ladder optimisation driven by a perceptual quality metric such as VMAF. Encoding a short probe of the

source at several operating points and selecting those on the convex hull of the quality-versus-bitrate curve would reduce delivery bandwidth at unchanged perceived quality, which lowers the largest remaining cost component of the system.

Chunked transcoding is in place, so the remaining work on long videos concerns its efficiency. Each chunk pays a container start-up of tens of seconds, which is why doubling the capacity helps a 25-minute video so little. Encoding several chunks in one longer-lived worker, or raising the minimum chunk length, would trade parallelism for less overhead, and the measurements of Section 6.5 suggest that the best choice depends on both the length of the video and the quota available. A related extension is to compare a faster x264 preset with the current one using VMAF on difficult clips, since a faster preset could shorten every chunk if the quality loss is acceptable.

Hardware acceleration offers a further avenue. Research on GPU-enabled serverless platforms for HEVC encoding [12] suggests substantial throughput gains, and adopting a more efficient codec such as HEVC or AV1 alongside H.264 would reduce bandwidth for clients that support it.

Finally, moving the rate limiting and deduplication state into a shared store, and adding distributed tracing across the S3, SQS, Lambda, and Batch boundaries, would prepare the system for horizontal scaling and make failures in the asynchronous pipeline substantially easier to diagnose.

Bibliography

[1] Netflix Technology Blog, "Rebuilding netflix video processing pipeline with microservices." https://netflixtechblog.com/rebuilding-netflix-video-processing-pipeline-with-microservices-4e5e6310e359, 2024. Accessed: 2026-08-10.

[2] Netflix Technology Blog, "The making of VES: the cosmos microservice for netflix video encoding." https://netflixtechblog.com/the-making-of-ves-the-cosmos-microservice-for-netflix-video-encoding-946b9b3cd300, 2024. Accessed: 2026-08-10.

[3] Amazon Web Services, "Video on demand on AWS: Implementation guide." https://docs.aws.amazon.com/solutions/latest/video-on-demand-on-aws/, 2025. Accessed: 2026-08-10.

[4] M. Zhang, Y. Zhu, C. Zhang, and J. Liu, "Video processing with serverless computing: A measurement study," in *Proceedings of the 29th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2019.

[5] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Computing Surveys*, vol. 57, no. 3, 2025. Published version of arXiv:2310.08437.

[6] Netflix Technology Blog, "Per-title encode optimization." https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2, 2015. Accessed: 2026-08-10.

[7] Netflix Technology Blog, "Dynamic optimizer: A perceptual video encoding optimization framework." https://netflixtechblog.com/dynamic-optimizer-a-perceptual-video-encoding-optimization-framework-e19f1e3a277f, 2018. Accessed: 2026-08-10.

[8] Amazon Web Services, "Serve private content with signed URLs and signed cookies: Amazon cloudfront developer guide." https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html, 2025. Accessed: 2026-08-10.

[9] M. Burrows, "The chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.

[10] S. Sankaraiah, L. H. Shuan, C. Eswaran, and J. Abdullah, "Scalable video encoding with macroblock-level parallelism," *EURASIP Journal on Advances in Signal Processing*, vol. 2014, p. 145, 2014.

[11] C.-Y. Chen, S.-C. Cheng, *et al.*, "Task-based parallel H.264 video encoding for explicit communication architectures," in *Proceedings of the International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*, 2011.

[12] A. Salcedo-Navarro, R. Peña-Ortiz, J. M. Claver, M. Garcia-Pineda, and J. Gutierrez-Aguado, "Towards GPU-enabled serverless cloud edge platforms for accelerating HEVC video coding," *Cluster Computing*, vol. 28, p. 68, 2025.

Appendix A

Supplementary Material

A.1 Repository Structure

The source code is organised as a monorepo with the following top-level layout.

```
DACNTT_VideoStreaming/
  frontend/            React single page application (Vite, HLS.js)
  backend/             Node.js Express API and test suite
  transcoder/          FFmpeg worker and Docker image
  infrastructure/
    modules/           Eleven reusable Terraform modules
    environments/      dev and prod instantiations
  .github/workflows/   Seven CI/CD workflows
  scripts/             Benchmarking and load testing scripts
  docs/                Architecture diagrams, cost analysis,
   measured results
  report/              LaTeX sources of this report
```

A.2 FFmpeg Transcoding Command

The following invocation produces all three renditions from one read of the source. A single input is followed by three output blocks, each carrying its own scaling filter and encoder settings, so FFmpeg decodes the source once and feeds all three filter chains from it. The second and third blocks are abbreviated here; in the source they repeat the first block in full. The master playlist is not written by FFmpeg: the worker generates it afterwards, which is why no `-master_pl_name` argument appears. It does so from the encoded output rather than from the settings above, measuring each rendition's peak

segment bit rate from the segment sizes and their `#EXTINF` durations, and reading the codec string for each variant out of the sequence parameter set of its first segment.

The keyframe interval is derived from the source rather than assumed. The worker probes the first video stream and reads `avg_frame_rate`, preferring it over `r_frame_rate` because the latter is a base rate FFmpeg infers from the timing grid and need not describe the frames that are actually present.

A screen recording taken from the project's own test material shows how far apart the two can be. The file reports `r_frame_rate` as 60000/1001, that is 59.94 fps, while `avg_frame_rate` reports 28.725 fps; counting frames confirms the second, at 287 frames in ten seconds. Every interval between consecutive frames turns out to be an integer multiple of 1/144 second — four, five or six ticks, giving instantaneous rates of 36, 28.8 and 24 fps — so the real timing grid is the 144 Hz display, and neither the 59.94 that `r_frame_rate` advertises nor the 30 the source game was capped at. The cap cannot be met exactly, because 144 is not divisible by 30 and a frame would have to occupy 4.8 ticks.

The consequence for the group of pictures is direct. Taking `avg_frame_rate` gives 172 frames, or 5.988 seconds, which fits inside the segment. Taking `r_frame_rate` would give 359 frames, or 12.5 seconds, more than twice the segment length. A fixed 30 fps assumption would give 180 frames, or 6.27 seconds, which also overruns. This is also why `-force_key_frames` is retained alongside `-g`: when the source rate varies between 24 and 36 fps, an interval counted in frames cannot correspond to a fixed interval in time, and only the time-based expression can.

Once the rate is known the group of pictures is `floor(fps × 6)` frames, floored rather than rounded so that it never exceeds the segment length. The same count of 180 frames illustrates why the rate has to be right before the arithmetic can be: it lasts 6.27 seconds at the 28.725 fps just measured, and 6.006 seconds at 29.97 fps, where it arises instead from rounding 179.82 upward. Both overrun a six-second segment, and in both cases the boundary drifts past the point where a keyframe exists. The value shown below, 144, is what a 24 fps source produces.

Two further arguments keep the boundaries aligned. Scene detection is disabled, because letting the encoder place keyframes at scene changes would allow the three renditions to cut at different points, and RFC 8216 requires that matching content in variant streams carry matching timestamps. `-force_key_frames` then pins a keyframe to every sixth second of media time; unlike `-g`, which only bounds the interval from above, it is absolute and independent of the detected frame rate, so a misdetected source still yields aligned segments.

The H.264 profile is pinned but the level is not. Each rendition carries a different frame size, and the lowest conforming level therefore differs between them, so the level is left for the encoder to derive from the resolution rather than fixed in the command.

```
ffmpeg -i input.mp4 -hide_banner -loglevel warning -y \
  -vf "scale=640:360:force_original_aspect_ratio=decrease,␣\
␣␣␣␣␣␣␣␣␣pad=640:360:(ow-iw)/2:(oh-ih)/2" \
  -c:v libx264 -b:v 400k  -maxrate 500k  -bufsize 800k  \
  -preset fast -profile:v main \
  -g 144 -keyint_min 144 -sc_threshold 0 \
  -force_key_frames "expr:gte(t,n_forced*6)" \
  -c:a aac -b:a 64k  -ar 44100 -ac 2 \
  -f hls -hls_time 6 -hls_list_size 0 \
  -hls_segment_filename 360p/segment_%03d.ts  360p/playlist.m3u8 \
  \
  -vf "scale=1280:720:..."  -c:v libx264 -b:v 1500k -maxrate 2000k
   -bufsize 3000k \
  -c:a aac -b:a 128k -f hls -hls_time 6 \
  -hls_segment_filename 720p/segment_%03d.ts  720p/playlist.m3u8 \
  \
  -vf "scale=1920:1080:..." -c:v libx264 -b:v 4000k -maxrate 5000k
   -bufsize 8000k \
  -c:a aac -b:a 192k -f hls -hls_time 6 \
  -hls_segment_filename 1080p/segment_%03d.ts 1080p/playlist.m3u8
```

A.3 Signed Cookie Custom Policy

The policy signed by the backend before issuing playback cookies. The wildcard in the resource path covers the manifest and every segment of one specific video, and nothing beyond it.

```
{
  "Statement": [
    {
      "Resource": "https://cdn.zelostech.site/videos/<videoId>/*",
      "Condition": {
        "DateLessThan": { "AWS:EpochTime": 1786300347 }
      }
    }
  ]
```

```
}
```

A.4 Reproducing the Measurements

The experimental results reported in Chapter 6 are produced by the scripts listed below. Detailed instructions, including the environment variables required and the order in which the measurements must be taken, are given in `docs/results/README.md` within the repository.

Every table of measurements in Chapter 6 is listed below against the script that produced it and the file its figures were read from, so that any individual number can be traced back to its source. Paths are relative to the repository root, and the result files are those under `docs/results/`.

Table A.1: Provenance of each measurement table

TABLE	SCRIPT	RESULT FILE
6.1	`scripts/benchmark-qoe.js`	`qoe-ttff.json`
6.2	`scripts/benchmark-qoe.js`	`qoe-ttff-cloudfront.json`
6.3	`scripts/benchmark-qoe.js`, deployed as a Lambda to each region	`qoe-ttff-multiregion.json`
6.4	`scripts/qoe/collect.js`	`qoe-playback-<profile>.json`
6.5	Segment sizes and `#EXTINF` durations read from the processed bucket	—
6.6	`scripts/qoe/collect.js`, run before and after the correction	`qoe-playback-<profile>.json`
6.7	`scripts/qoe/compare.js`, driven by `docs/results/qoe-sources.json`	`qoe-playback-<source>-<profile>.json`, `qoe-comparison.json`
6.8	`scripts/qoe/collect.js` with `-dpr`	`qoe-playback-<source>-unthrottled.json`, `qoe-playback-<source>-unthrottled-dpr2.json`
6.9	`scripts/measure-transcode.js`	`transcode-timing.json`
6.15	`scripts/k6-load-test.js`	`k6-summary.json`, `stress-test-concurrency.json`
6.16	`scripts/node-load-test.js`	`node-load-test.json`
6.17	`scripts/k6-load-test.js`	`drain-rate-real-payload.json`, `stress-test-concurrency.json`
6.18	Published AWS list prices, computed analytically	—

A.5 Deploying the Infrastructure

The complete AWS environment is provisioned from a single directory.

```
cd infrastructure/environments/dev
terraform init
terraform apply
```

Enabling signed cookies additionally requires generating an RSA key pair, supplying the public key to Terraform, and configuring the backend with the corresponding private key and the key pair identifier reported by `terraform output`. The full procedure is documented in `docs/PRIVATE_VIDEO_SIGNED_COOKIES.md`.